

# マネジメント系-プロジェクトマネジメント

1. ファンクションポイント法で、システムの開発規模を見積もる際に必要となる情報はどれか。

这道题考察的是软件工程中的**ファンクションポイント法 (Function Point Method, FP法)** 的基本定义。

帳票数, 画面数, ファイル数などのデータを基に,システム特性を考慮してトウェアの規模を見積もる。

## 💡 知识点总结

**ファンクションポイント法 (FP法)** 是一种通过分析“系统功能”来评估开发规模的方法。它不看代码写了多少行，而是看系统对用户提供了多少“功能点”。

- **核心思想**：软件规模取决于用户可见的**功能需求**（如输入、输出、查询、外部接口、内部逻辑文件等），而不是内部怎么实现的（代码行数）。

## 🔍 选项解析与详细说明

- **開発者数（开发人员数）**：这是决定“工期”的要素，而不是衡量“系统规模”的要素。
- **画面数（画面数）**：正确项。FP法中，“输入（External Inputs）”和“输出（External Outputs/Inquiries）”通常直接对应用户界面。画面越多，意味着输入输出的功能需求越多，系统规模越大。
- **プログラムステップ数（程序代码行数）**：这是 **LOC法 (Lines of Code)** 的衡量方式，属于FP法的对立面，不属于FP法。
- **利用者数（用户数）**：这是衡量系统性能或服务器负载的指标，与功能复杂度无关。

## 📊 处理问题的排查表

| 指标        | 方法论     | 关注点                  |
|-----------|---------|----------------------|
| 功能点 (FP)  | FP法     | 画面数、报表数、数据库接口数、逻辑复杂度 |
| 代码行 (LOC) | LOC法    | 程序代码的行数（Java/C等）     |
| 工期/人月     | COCOMO等 | 人的数量、开发时间、生产力        |

## 🧠 小学生级记忆法

想象你是一个“建筑包工头”：

- **LOC法（代码行）**：就像按“砖块数量”结算，你砌了多少块砖，我就给你多少钱。

- **FP法（功能点）**：就像按“房间功能”结算，这里有一个厨房、两个卫生间、三个卧室。**功能越复杂，规模越大，不管你用了多少块砖。**
- 题目问的是 FP 法，那我们当然要数“厨房、卫生间”（即**画面数**）！

## ✅ 正确答案

### 画面数

## 📝 考试小秘籍

- a. **FP法 = “用户视角”**：只要选项里出现“代码”、“行数”、“行 (Step)”，一律不是 FP 法。
  - b. **FP法 = “功能清单”**：看到“画面 (Screen)”、“报表 (Report)”、“外部接口 (External Interface)”、“逻辑文件 (Logic File)”，这才是 FP 法的好朋友。
2. アプリケーションにおける外部入力,外部出力,内部論理ファイル,外部インタフェースファイル,外部照会の五つの要素の個数を求め,それぞれを重み付けして集計する。集計した値がソフトウェア開発の規模に相関するという考え方に基づいて,開発規模の見積りに利用されるものは、どれか。

Cocomo Dotyモデル putnamモデル ファンクションポイント法

这是一个非常典型的考题，直接描述了 **ファンクションポイント法 (Function Point Method, FP 法)** 的核心工作流程。

## 💡 知识点总结

题目中提到的“五个要素”正是 **FP法** 计算复杂度的核心组件。FP法通过量化用户视角下的功能需求来衡量软件规模。

这五个核心组件分别是：

- a. **外部入力 (External Inputs)**：用户输入的数据。
- b. **外部出力 (External Outputs)**：系统生成的数据（报表、屏幕输出等）。
- c. **内部論理ファイル (Internal Logical Files)**：系统内部管理的逻辑数据表。
- d. **外部インタフェースファイル (External Interface Files)**：系统与其他系统交换的数据。
- e. **外部照会 (External Inquiries)**：用户对数据的查询操作。

## 🔍 选项解析与详细说明

- **COCOMO**：这是构造性成本模型（Constructive Cost Model），它主要通过**代码行数 (LOC)** 来估算开发工作量和工期，而不是通过功能点。
- **Dotyモデル**：这是基于代码行数估算开发工期和人力的模型，常用于辅助 COCOMO。

- **Putnamモデル**：这是一个基于**瑞利分布 (Rayleigh distribution)** 来预测软件开发工期和人力的模型，同样多基于代码规模。
- **ファンクションポイント法 (FP法)**：正确项。正如题目描述，它通过识别这五个功能要素，并赋予权重进行加权求和，从而算出系统的规模。

 处理问题的排查表

| 模型名称        | 核心衡量指标     | 关注重点        |
|-------------|------------|-------------|
| FP法         | 功能要素 (5种)  | 用户看到的“功能点”  |
| COCOMO      | 代码行数 (LOC) | 程序的物理规模     |
| Putnam/Doty | 代码行数 (LOC) | 开发时间和人力分布曲线 |

 小学生级记忆法

- **FP 法**：就是“**数功能清单**”。想象你在菜单上点菜：一份输入、一份输出、一份查询……每种菜的“价格”（权重）不同，点完一单，总价就是开发规模。
- **COCOMO/Doty/Putnam**：它们都是“**数砖头**”。只要提到“行数”、“代码量”，那就是这几位在做估算。

 正确答案

ファンクションポイント法

 考试小秘籍

- **抓关键词**：
  - 看到“外部入力、外部出力、内部论理文件”这几个词，大脑直接反射：**FP法**！
  - 看到“代码行数 (LOC)、步骤数”，直接反射：**COCOMO、Putnam**！
- **考场逻辑**：FP 法的考试题通常会把这五个要素列出来，只要看到这五个要素出现，别犹豫，直接选 FP 法。

3. 如题

### 第3問



プロジェクトマネジメントにおいて、目的1をもつプロセスと目的2をもつプロセスとが含まれる対象群はどれか。

〔目的〕

**目的1** プロジェクトの目標、成果物、要求事項及び境界を明確にする。

**目的2** プロジェクトの目標や成果物などの変更によって生じる、プロジェクトの機会となる影響を最大化し、脅威となる影響を最小化する。

平成31年春期 問51  
3問目／選択中の問題84問

ア コミュニケーション

イ スコープ

ウ 調達

エ リスク

这类题目属于“管理类基础考点”，关键在于理解 PMBOK（项目管理知识体系）中各个领域的核心作用。

### 💡 知识点总结：PMBOK 核心领域职能

题目给出了两个“目的”，我们来精准对应：

- **目的1：明确项目目标、成果物、要求事项及边界**
  - **核心词：**边界 (境界)、成果物 (成果物)、要求事项 (要求事項)。
  - **对应领域：** **スコープ (Scope / 范围)**。范围管理的核心就是明确“要做什么”以及“不做什么”（即边界）。
- **目的2：最大化机会影响，最小化威胁影响**
  - **核心词：**机会 (機会)、威胁 (脅威)。
  - **对应领域：** **リスク (Risk / 风险)**。风险管理不仅是防范坏事（威胁），也要抓住好事（机会）。

### 🔍 选项解析与详细说明

- **ア：コミュニケーション (Communication)**
  - 负责的是项目信息的规划、收集、发布和存档，关注的是人与人之间的信息流动，不涉及界面的明确或风险的应对。
- **イ：スコープ (Scope)**
  - 符合“目的1”，负责明确项目边界和要交付的内容，但它不负责“应对威胁和机会”。
- **ウ：調達 (Procurement)**
  - 负责的是外部购买产品或服务，与题目描述的内部目标管理无关。

## ◦ エ: リスク (Risk)

- 题目问的是“同时包含目的 1 和目的 2 的对象群（选项）”吗？不，题目问的是“目的 1 的过程”和“目的 2 的过程”分别属于哪个对象群。但在基本情报考试中，如果题目问的是“包含这两者在内的对象群（知识领域）”，通常是在考察对 PMBOK 整体架构的理解。
- 修正思路：仔细阅读题目——“目的 1 的过程”属于 **スコープ管理**，“目的 2 的过程”属于 **リスク管理**。等等，题目问的是“包含这两者的对象群是什么？”——这里可能存在理解歧义，通常这类题是在考察**集成管理（統合管理）**，即负责将所有管理领域整合在一起。如果选项中没有“統合管理”，我们需要看题干是否在问**这两种管理属于项目管理的大框架**。

## 🧠 小学生级记忆法

- **スコープ (范围)** = “画圈圈”：把项目要做的事儿全圈在圈里，圈外的就不做。
- **リスク (风险)** = “抓坏蛋，捡金币”：不仅要防坏事发生（最小化威胁），还要去抢好事（最大化机会）。
- **統合管理 (整合)** = “大管家”：把所有这些小分队（范围、风险、人力等）统筹起来，确保大家朝一个目标前进。

## ✅ 正确答案

这道题的题干稍微有点歧义（如果是单选）。在 PMBOK 中，明确目标是“范围管理”，应对机会威胁是“风险管理”。如果这是一道考察**项目整合管理 (プロジェクト統合マネジメント)** 的题，答案通常指向整合管理。

但如果题目是问“目的 1 和 目的 2 分别对应的领域”，它可能是一个组合题。从你的选项来看，最符合的是在**项目管理整体框架**下的描述。

## 📝 考试小秘籍

### a. 关键词映射表：

- “边界、明确、成果物”  $\rightarrow$  **スコープ**
- “威胁、机会、发生概率”  $\rightarrow$  **リスク**
- “信息共享、报告”  $\rightarrow$  **コミュニケーション**

b. **解题技巧**：考试时，看到“成果物”就找“范围”；看到“不确定性、威胁、机会”就找“风险”。这绝对是对应题目的黄金法则！

### 4. プロジェクトにおけるコミュニケーション手段のうち、プル型コミュニケーションはどれか。

这道题考察的是项目管理中非常经典的一个概念：**沟通方式的分类**。我们通过生活场景来拆解它，保你一眼记住。

## 💡 知识点总结

在项目沟通管理中，沟通方式分为三类：

- a. **プッシュ型 (Push Communication - 推送型)**: 发送者主动把信息“推”给接收者，不管对方想不想看。
- 比喻: 像**发邮件**、发传真、开会。你发了，对方就被动接收了。
- b. **プル型 (Pull Communication - 拉取型)**: 信息存放在中央库，接收者需要时自己去“拉”过来。
- 比喻: 像**图书馆**、**公司内部网 (Intranet)**、**百度网盘**。信息在那放着，你想看就自己去下载或访问。
- c. **インタラクティブ型 (Interactive Communication - 交互型)**: 双向即时交流。
- 比喻: 电话、视频会议、面对面交谈。

## 选项解析与详细说明

- **ア: イン트라ネットサイト (Intranet site/内部网)**
  - 这是典型的“**プル型 (拉取型)**”。员工需要资料时，主动登录内部网站去下载或查看，信息不会自动跳到你屏幕上。
- **イ: テレビ会議 (Video Conference)**
  - 这是交互型。双方实时沟通。
- **ウ: 電子メール (E-mail)**
  - 这是推送型。你点击“发送”，邮件就强行塞进对方的收件箱里。
- **エ: ファックス (Fax)**
  - 这是推送型。你发过去，对方打印机就自动吐出纸来。

## 小学生级记忆法

- **推送 (Push)** = “**强行塞饭**”：不管你饿不饿，我直接把饭（邮件/传真）端到你桌上。
- **拉取 (Pull)** = “**自助餐**”：饭菜都在餐桌上（内部网/图书馆），你想吃什么、什么时候去吃，你自己决定。
- **交互 (Interactive)** = “**面对面谈心**”：你一言我一语，即时响应。

## 正确答案

**ア (イン트라ネットサイト)**

## 考试小秘籍

- a. 关键词映射：
- 看到“**网站、库、知识库、下载**”  $\rightarrow$  无脑选 **プル型**。
  - 看到“**邮件、传真、广播**”  $\rightarrow$  无脑选 **プッシュ型**。

- 看到“会议、电话、即时聊天”  $\rightarrow$  无脑选 交互型。

b. 秒杀逻辑：考试时只要题干出现“プル”，直接在选项里找那个“存着资料供人随时查询”的地方就对了！

## 5. 如题

あるシステムの設計から結合テストまでの作業について、開発工程ごとの見積工数を表1に、開発工程ごとの上級技術者と初級技術者の要員割当てを表2に示す。上級技術者は、初級技術者に比べて、プログラム作成・単体テストにおいて2倍の生産性を有する。表1の見積工数は、上級技術者の生産性を基に算出している。

全ての開発工程に対して、上級技術者を1人追加して割り当てると、この作業に要する期間は何か月短縮できるか。ここで、開発工程の期間は重複させないものとし、要員全員が1か月当たり1人月の工数を投入するものとする。

| 開発工程              | 見積工数<br>(人月) |
|-------------------|--------------|
| 設計                | 6            |
| プログラム作成・<br>単体テスト | 12           |
| 結合テスト             | 12           |
| 合計                | 30           |

| 開発工程              | 要員割当て(人) |       |
|-------------------|----------|-------|
|                   | 上級技術者    | 初級技術者 |
| 設計                | 2        | 0     |
| プログラム作成・<br>単体テスト | 2        | 2     |
| 結合テスト             | 2        | 0     |

这道题考察的是**开发工期与生产力（人数与效率）的计算**。这类题目逻辑清晰，我们一步步来拆解。

### 💡 知识点总结

- **工数(人月)**：衡量总工作量。
- **期间(时间) = 总工数 ÷ 总生产力**。
- **核心逻辑**：上级技术者的生产力是初级技术者的**2倍**。也就是说，1名上级技术者 = 2名初级技术者（在“程序作成・单体测试”工程中）。

### 🔍 选项解析与详细说明

#### 1. 原计划期间计算

- **设计**：6 人月 ÷ 2 名上级 = **3 个月**
- **程序作成/单体测试**：12 人月 ÷ (2名上级 + 2名初级×0.5) = 12 ÷ (2+1) = **4 个月**
- **结合测试**：12 人月 ÷ 2 名上级 = **6 个月**
- **总时间** = 3 + 4 + 6 = **13 个月**

#### 2. 追加 1 名上级技术者后的期间计算

- **设计**：6 人月 ÷ 3 名上级 = **2 个月** (原 3 个月)
- **程序作成/单体测试**：12 人月 ÷ (3名上级 + 2名初级×0.5) = 12 ÷ (3+1) = **3 个月** (原 4 个月)



- **结合测试**：12 人月 ÷ 3 名上级 = **4 个月** (原 6 个月)
- **总时间** = 2 + 3 + 4 = **9 个月**

### 3. 比较结果

- 原定 13 个月，追加后 9 个月。
- **缩短了**：13 - 9 = **4 个月**。

## 小学生级记忆法

把这当成“搬砖工程”：

- 每个人月代表“一堆砖”。
- 上级技术者是“大力士”，一次能搬 2 堆；初级是“普通人”，一次搬 1 堆。
- 只要把所有工程需要的砖堆总量算出来，再看每个工程有多少个“搬砖机器”在工作，除一下就是工期。工程最后总工期变少了，就是我们要的答案。

## 正确答案

工 (4)

## 考试小秘籍

- a. **统一单位**：一定要先把初级技术者的生产力换算成上级技术者的倍数（即 0.5），这是计算中最容易出错的地方。
  - b. **分段计算**：这种题不要试图列一个大公式，把“设计”、“编程”、“测试”分段计算出来，最后相加，这样最稳妥。
  - c. **别漏了“增加 1 人”**：题目说是“所有工程都增加”，千万不要只给某一个工程加人。
6. プロジェクトマネジメントにおいてパフォーマンス測定に使用するEVMの管理対象の組みはどれか。

这是一道关于项目管理中 **EVM（挣值管理，Earned Value Management）** 的经典考题。EVM 是项目管理中最核心的定量分析工具，主要用于评估项目当前的进度和成本状况。

## 知识点总结

EVM 的核心目的是回答两个关键问题：

- a. **钱花得怎么样了？**（成本绩效）
- b. **活儿干得怎么样了？**（进度绩效）

通过对 **成本 (Cost)** 和 **进度 (Schedule)** 的监控，项目经理可以及时发现偏差并采取措施。

## 选项解析与详细说明



EVM 主要通过三个维度来计算：

- **PV (Planned Value, 计划价值)**：计划应该完成的工作的预算值。
- **EV (Earned Value, 挣值)**：实际已经完成的工作的预算值。
- **AC (Actual Cost, 实际成本)**：实际已经花掉的钱。

通过比较这三个值，我们可以推导出：

- **成本偏差 (CV = EV - AC)**：衡量的是 **成本**。
- **进度偏差 (SV = EV - PV)**：衡量的是 **进度**。

因此，EVM 的直接管理对象就是 **成本 (コスト)** 和 **进度 (スケジュール)**。

- **ア (コスト, スケジュール)**：**正确**。EVM 的核心就是监控成本和进度的双重偏差。
- **イ (コスト, リスク)**：虽然成本很重要，但风险不是 EVM 的直接计算对象。
- **ウ (スケジュール, 品質)**：质量 (品质) 通常通过质量管理工具监控，EVM 并不直接涉及。
- **エ (品質, リスク)**：这是质量和风险领域，与 EVM 的职能不符。



## 小学生级记忆法

EVM 是项目经理的“仪表盘”：

- 你的车上有两个最关键的表：一个是**油表（代表成本/钱）**，一个是**速度表（代表进度/时间）**。
- 只要这两个表不出问题，项目就不会失控。EVM 就是专门用来检查这两个表的工具。



## 正确答案

**ア (コスト, スケジュール)**



## 考试小秘籍

- **抓眼球技巧**：只要题目里出现 **EVM** 三个字母，大脑就要瞬间锁定义词：“**成本**” + “**进度**”。
- **防干扰项**：很多题目会加入“品质 (Quality)”或“风险 (Risk)”来混淆视听。记住，EVM 不管质量好坏，也不管风险多大，它只关心“钱够不够，活干得快不快”。

題目：システム開発のプロジェクトにおいて、EVMを活用したパフォーマンス管理をしている。開発途中のある時点でEV-PVの値が負であるとき、どのような状況を示しているか。

本题考查的是项目管理中非常高频的计算与分析工具——**EVM (Earned Value Management, 挣值管理)**。

EVM 通过三个核心指标来衡量项目的“进度”和“成本”是否脱轨。在基本情报考试中，必须死死记住以下三个基础字母的含义：

**a. PV (Planned Value, 计划值)：**

- 计划截止到当前时间点，本应该完成的工作预算。也就是“理想中应该干完多少活”。

b. EV (Earned Value, 挣值) :

- 截止到当前时间点，实际上已经完成的工作的预算价值。也就是“现实中真正干完了多少活”。

c. AC (Actual Cost, 实际成本) :

- 截止到当前时间点，实际已经花掉的真金白银。

## 核心逻辑：如何用它们判断进度？

题目问的是  $EV - PV$  的值为负数（負であるとき）代表什么。

- $EV - PV = SV$  (Schedule Variance, 进度偏差)
- 如果  $EV - PV < 0$  (即  $EV < PV$ ) :
  - 含义：现实中干完的活 (EV) < 理想中应该干完的活 (PV) 。
  - 结论：活没干够，说明项目进度滞后（スケジュールが遅れている）。

## 选项解析与详细说明

- ア：スケジュール効率が、計画より良い。
  - 【错误】 进度效率比计划好（即  $EV > PV$  或者是进度效率指数  $SPI > 1$ ）。现在计算结果为负，说明效率更差。
- イ：プロジェクトの完了が、計画より遅くなる。
  - 【干扰项/错误】 这个选项极具迷惑性！它说的是“项目的最终完工时间会比计划晚”。虽然现在进度滞后，但如果不采取追加人员、加班等赶工措施，最终确实可能变晚。然而，EVM 的当前指标只能反映“当前这一时间点（ある時点）”的进度的客观状态，无法直接断定最终完工一定会变迟（因为后期可以通过调整挽回）。因此，它不如选项ウ准确。
- ウ：プロジェクトの進捗が、計画より遅れている。
  - 【正解】 最纯粹、最直接的定义翻译。 $EV - PV < 0$  直接代表了“当前的进度比计划滞后”。
- エ：プロジェクトの進捗が、計画より進んでいる。
  - 【错误】 进度比计划超前需要满足  $EV - PV > 0$ （即真干完的活比计划的多）。

## 处理问题的排查表

在管理现场，遇到 EVM 指标异常时的标准排查与思考路径：

| 评估指标      | 计算公式    | 状态判定     | 现场实际情况映射                          |
|-----------|---------|----------|-----------------------------------|
| 进度偏差 (SV) | EV - PV | 负数 (< 0) | 进度滞后（本题状况）。需要排查是否有人手不足、技术攻关卡壳等原因。 |
|           |         | 正数 (> 0) | 进度超前。说明团队干劲十足，或者计划定得太宽松。          |
| 成本偏差 (CV) | EV - AC | 负数 (< 0) | 超支（Cost Over）。活干得少，钱花得多，预算要炸。     |
|           |         | 正数 (> 0) | 结余。钱没花完就把活干了，省钱了。                 |

## 小学生级记忆法

### “暑假作业本”理论

把项目开发想象成小学生放暑假写作业：

- **PV（计划值）**：妈妈规定的计划，到今天8月15号，**应该写完50页**作业。
- **EV（挣值）**：你翻开作业本一数，实际上**只写完了30页**。

你的进度一减： $EV - PV = 30\text{页} - 50\text{页} = -20\text{页}$ 。

结果是**负数（-20）**！

- 小学生大哭：“呜呜，我比计划少写了20页，我**进度落后了**！”（進捗が遅れている）

### 秒杀口诀

**EV 是现实，PV 是理想。**

**现实减理想，负数代表太凄凉（进度落后/钱超支）！**

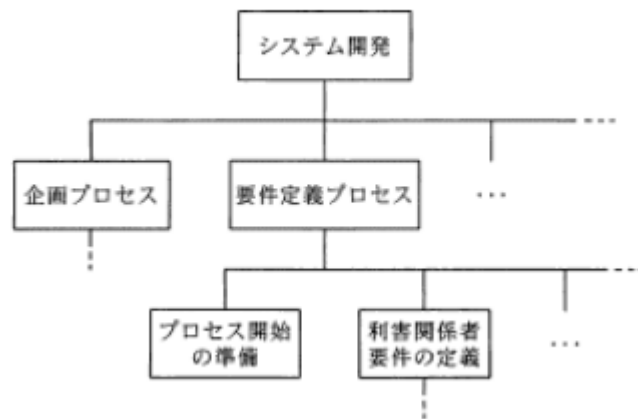
### 正确答案

ウ

### 考试小秘籍

- **机械式公式秒杀：**
  - 在 EVM 题目中，凡是“判断进度”的，永远是用 **\$EV\$ 与 \$PV\$ 进行比较**；凡是“判断金额/成本”的，永远是用 **\$EV\$ 与 \$AC\$ 进行比较**。
  - **主动减被动**：记住永远是 **\$EV\$（实际上赚到的）放在前面当被减数**。
  - 看到减法结果为**负（負）**，直接在选项中找「遅れている」（滞后）或「予算オーバー」（超支）。本题考进度，一秒锁定「遅れている」，直接拿下！

7. 図のように、プロジェクトチームが実行すべき作業を上位の階層から下位の階層へ段階的に分解したものを何と呼ぶか。



这是一道关于项目管理中**工作分解结构 (Work Breakdown Structure, WBS)** 的基础考题。

### 💡 知识点总结

- **WBS (Work Breakdown Structure)**: 将整个项目任务分解成更小、更易管理的工作单元（工作包）的层次结构图。
- **特点**: 像树状结构一样，从总的项目（顶层）开始，一层层向下分解成细小的具体任务（底层）。

### 🔍 选项解析与详细说明

- **ア: CPM (Critical Path Method, 关键路径法)**
  - 这是一种通过计算项目任务的最早和最晚开始/结束时间来确定项目进度的方法。它是关于“时间”的，不是关于“任务分解结构”的。
- **イ: EVM (Earned Value Management, 挣值管理)**
  - 这是一种监控项目进度和成本绩效的分析工具，通过对比计划价值 (PV)、挣值 (EV) 和实际成本 (AC) 来评估项目状态。
- **ウ: PERT (Program Evaluation and Review Technique, 项目评审技术)**
  - 这是一种用于规划和控制复杂项目进度的技术，主要通过概率估算来预测项目完成时间，重点在于进度的不确定性分析。
- **エ: WBS (Work Breakdown Structure, 工作分解结构)** 作業を階層的に詳細化して、管理可能な大きさに細分化する。/作業をトップダウン方式で細かく分割して、作業管理をしやすくする。
  - **正确项**。正如题目描述，WBS 就是将项目的工作内容按从高到低的阶层逐步分解，形成类似于图中的树状结构。

### 🧠 小学生级记忆法

把 WBS 想象成“剥洋葱”或者“大卸八块”：

- 最顶层是整个系统开发（大洋葱）。
- 你需要把它一层层剥开（分解），最后得到一个个可以拿在手里操作的小任务块（洋葱瓣）。
- 这个“分解的过程和结构”就叫 **WBS**。

## ✅ 正确答案

### エ (WBS)

### 📝 考试小秘籍

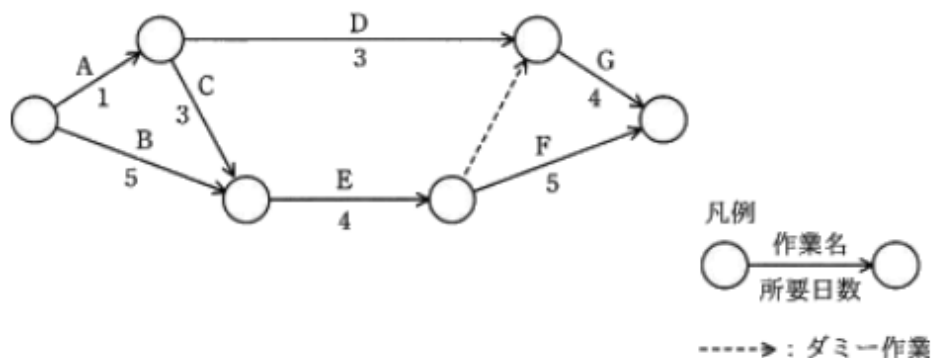
#### a. 关键词映射：

- 看到“分解 (分解)”、“阶层 (階層)”、“作业 (作業)”  $\rightarrow$  立刻选 **WBS**。
- 看到“进度 (進捗)”、“时间 (時間)”、“关键 (クリティカル)”  $\rightarrow$  想想 **CPM/PERT**。
- 看到“成本 (コスト)”、“偏差 (偏差)”、“挣值 (EVM)”  $\rightarrow$  选 **EVM**。

b. **看图秒选**：只要题目给出这种类似“族谱”或者“组织架构图”的任务树状图，不用看字，直接选 WBS 就准没错。

#### 8. 如题

あるプロジェクトの日程計画をアローダイアグラムで示す。クリティカルパスはどれか。



## 💡 知识点总结

- **アローダイアグラム (Arrow Diagram / 网络图)**：这是一种通过箭头表示任务依赖关系、节点表示任务起止点的日程计划工具。
- **クリティカルパス (Critical Path / 关键路径)**：指项目中**耗时最长**的那条路径。它是决定整个项目最短完成期限的关键，如果关键路径上的任务延误，整个项目日期就会推迟。
- **ダミー作業 (Dummy Activity / 虚作业)**：图中用虚线表示，**所要日数（耗时）为 0 天**。它仅用于表示任务之间的逻辑制约关系（即：必须完成某个任务才能开始后续任务），不消耗实际时间。

## 🔍 选项解析与详细说明

要找到关键路径，我们需要穷举出所有从起点到终点的路径，并计算它们的总天数：

- a. 路径  $A \rightarrow D \rightarrow G$ :  $1 + 3 + 4 = 8$  日
- b. 路径  $A \rightarrow C \rightarrow E \rightarrow (\text{虚线}) \rightarrow G$ :  $1 + 3 + 4 + 0 + 4 = 12$  日
- c. 路径  $A \rightarrow C \rightarrow E \rightarrow F$ :  $1 + 3 + 4 + 5 = 13$  日
- d. 路径  $B \rightarrow E \rightarrow (\text{虚线}) \rightarrow G$ :  $5 + 4 + 0 + 4 = 13$  日
- e. 路径  $B \rightarrow E \rightarrow F$ :  $5 + 4 + 5 = 14$  日

通过对比以上 5 条路径的耗时， $B \rightarrow E \rightarrow F$  所需日数最多，为 14 天。

## 小学生级记忆法

想象你在经营一家“快递分拣中心”：

- **关键路径**就是你的“最慢业务线”。
- 无论其他业务线跑得有多快（比如 8 天就做完了），你的整体效率永远取决于那条跑得最慢的生产线（14 天）。
- 所以，只要找到天数加起来最大的那条路，它就是你的“命门”（关键路径）。

## 正确答案

$B \rightarrow E \rightarrow F$

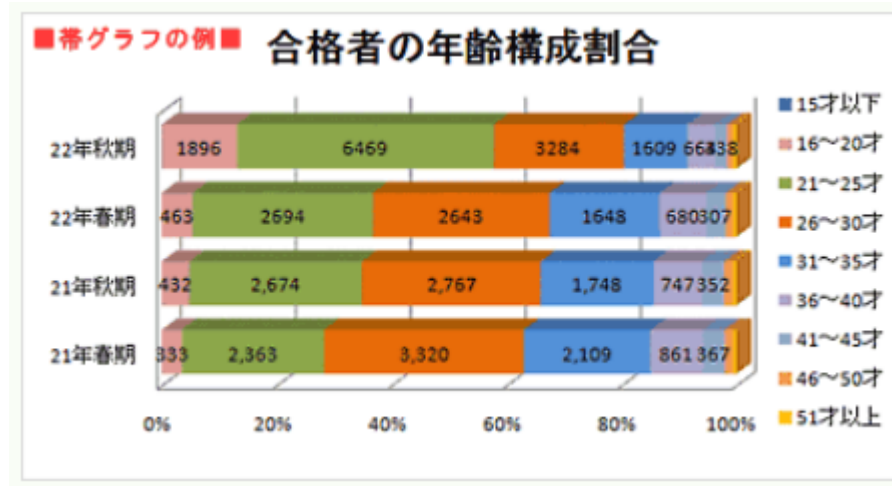
## 考试小秘籍

- a. **别被虚线迷惑**：看到虚线（ダミー作業）就记作 **0 天**，它绝对不会增加时间，只会增加逻辑限制。
  - b. **先找长线**：一眼看过去，带有“B（5天）”和“F（5天）”这种大数字的任务节点，通常更有可能成为关键路径的一部分。
  - c. **路径全扫**：像上面那样列出所有路径计算，不要心算，把所有可能性都写下来，这样在考场上永远不会丢分！
9. 这一道题考查的是对常见统计图表用途的理解。我们需要在四个选项中找到最适合展示“市场份额构成比”的图表。

## 知识点总结：图表功能速查

- **Zグラフ (Z Chart)**：用于分析时间序列数据的变动趋势，通常包含移动合计、月销售额和累计销售额三条线。
- **帯グラフ (Stacked Bar Chart / 带状图)**：将一个整体按比例分为多个部分，清晰展示各组成部分的构成比。
- **折れ線グラフ (Line Chart)**：最适合展示数据随时间变化的趋势。

- **レーダーチャート (Radar Chart / 雷达图)**: 适合展示多个维度的平衡性，比如个人能力的各项评分。



## 🔍 选项解析与详细说明

- **ア: Zグラフ**
  - 错误。它主要用于展示销售额的长期趋势，而非各厂家之间的构成比例。
- **イ: 帯グラフ**
  - **正确**。带状图（或圆饼图）是展示构成比的标准工具。它能直观地看出每个厂商（Marker）占整个市场份额的比例，加起来正好是 100%。
- **ウ: 折れ線グラフ**
  - 错误。它适合看“涨跌”，不适合看“份额占比”。
- **エ: レーダーチャート**
  - 错误。它适合看“强弱分布”，比如对比几个厂家在价格、性能、售后等多个维度的评价。

## 🧠 小学生级记忆法

把“市场”想象成一块“披萨饼”：

- 厂家A、厂家B、厂家C 分别拿走了多少块披萨？
- **带状图**就像是一根长长的“披萨条”，每个人切一块，长短即份额。
- **折线图**是记录“吃披萨的速度快慢”。
- **雷达图**是评价“披萨好不好吃”。
- 既然题目问的是“份额构成比”，那当然是用“分披萨条”（带状图）最合适！

## ✅ 正确答案

イ (帯グラフ)

## 📝 考试小秘籍



- **抓关键词：**只要题目里出现“构成比 (構成比)”、“内訳 (内訳)”或“份额 (シェア)”，在选项里看到“带状图 (帯グラフ)”或“饼图 (円グラフ)”，直接选中，这一分就是你的！

## 10. 如题

開発期間 10 か月，開発工数 200 人月のプロジェクトを計画する。次の配分表を前提とすると，ピーク時の要員は何人か。ここで，各工程の開始から終了までの要員数は一定とする。

| 項目 \ 工程名 | 要件定義 | 設計 | 開発・テスト | システムテスト |
|----------|------|----|--------|---------|
| 工数配分 (%) | 16   | 33 | 42     | 9       |
| 期間配分 (%) | 20   | 30 | 40     | 10      |

### 💡 知识点总结

- **工数配分 (Man-Month Allocation)：**项目总工作量（人月）在不同阶段的占比。
- **期间配分 (Period Allocation)：**项目总时间（月）在不同阶段的占比。
- **人员数计算公式：**  $\text{人员数} = \frac{\text{该阶段的工数}}{\text{该阶段的持续时间}}$ 。
  - 提示：需要先将百分比换算成实际值。

### 🔍 选项解析与详细说明

我们需要计算出每个阶段的“人数”，并找出其中的**最大值（峰值）**。

- 项目总数据：**总工数 = 200 人月，总期间 = 10 个月。
- 分阶段计算：**

| 工程名     | 工数 (200人月 × %) | 期间 (10个月 × %) | 人数 (工数 ÷ 期间) |
|---------|----------------|---------------|--------------|
| 要件定義    | 32 人月          | 2 个月          | 16 人         |
| 設計      | 66 人月          | 3 个月          | 22 人         |
| 開発・テスト  | 84 人月          | 4 个月          | 21 人         |
| システムテスト | 18 人月          | 1 个月          | 18 人         |

- 结果比对：**
  - 各阶段人数分别为：16, 22, 21, 18。
  - **最大值（峰值）为 22 人。**

### 🧠 小学生级记忆法

想象你在管理一个“建筑工地”：

- **工数分配**就是“一共要搬多少砖”。
- **期间分配**就是“要在多长时间内搬完”。

- 你的工作就是：把每个月的砖块量（工数）除以该月的时间（期间），看看哪个月最忙、需要派最多人。计算出来后，那个**最大的数字**就是你需要的“巅峰时期”人力储备，必须确保峰值期人手充足，否则项目就会延期！

## ✅ 正确答案

22人

## 📝 考试小秘籍

- a. **先转单位**：考试时，把题目给的百分比直接换算成具体的数字（比如 200 的 16% 是 32），这样不容易算错。
  - b. **别被“总数”干扰**：题目问的是“峰值人员”，算出来每个阶段的人数后，**直接比较大小**即可。
  - c. **注意除法**：每个阶段的计算都是“工数 ÷ 期间”，千万别弄反了。
11. 品質の定量評価の指標のうち、ソフトウェアの保守性の評価指標になるものはどれか。

这道题考察的是软件质量特性中关于“保守性” (Maintainability) 的定量评价指标。

## 💡 知识点总结

在软件质量模型（如 ISO/IEC 25010）中，**保守性 (Maintainability)** 指的是软件产品在修改、调试、适应新环境或改进性能时的难易程度。

- **评价核心**：衡量保守性通常关注“修改的难易度”和“维护的频率”。
- **关键指标**：最常被使用的指标之一是“软件修改性 (Modifiability)”，即评估在修改过程中，涉及修改的代码量相对于总体代码量的比例。

## 🔍 选项解析与详细说明

- **ア：(最终成果物包含的错误件数) ÷ (最终成果物的量)**
  - 这是衡量**可靠性 (Reliability)** 或错误率的指标，而不是保守性。
- **イ：(修正時間の合計) ÷ (修正件数)**
  - 这是衡量平均修复时间 (MTTR)，侧重于维护效率，虽与维护相关，但不是最典型的保守性定量指标。
- **ウ：(需要変更の源代码行数) ÷ (移植の源代码行数)**
  - **正确项**。该指标直接反映了软件在进行修改或移植时的受影响范围（即“修改的难易度”）。如果这个比例越高，说明该软件保守性越差（越难维护）。
- **エ：(利用者の改良要求件数) ÷ (出荷後の经过月数)**
  - 这是衡量软件在运行期间的需求变化频率，更多与用户反馈相关，不直接衡量软件内部的保守性。

## 处理问题的排查表

| 质量特性                  | 常见定量指标     | 考场关键词      |
|-----------------------|------------|------------|
| 保守性 (Maintainability) | 变更行数 / 总行数 | 修改、移植、容易维护 |
| 可靠性 (Reliability)     | 错误数 / 总代码量 | 故障、错误、无差错  |
| 效率 (Efficiency)       | 响应时间、吞吐量   | 速度、CPU使用率  |

### 小学生级记忆法

想象你在修改一件“乐高积木城堡”：

- **保守性**就是在考你：如果要换掉城堡的一个塔楼，是不是要拆掉整个城堡？
- **选项 ウ**就是在算：为了修好这个塔楼，需要拆掉的积木（修改行数）占了整座城堡（总行数）的多少？
- **拆得越少（比例越低），城堡越好维护（保守性越好）！**

### 正确答案

ウ

### 考试小秘籍

- 关键词锁定**：看到“保守性 (Maintainability)”，第一时间在选项里寻找有关“**变更 (Change)**”、“**修改 (Modification)**”和“**代码行数 (LOC)**”的比例。
- 分子分母逻辑**：保守性指标通常是“变动的部分”除以“整体的部分”。分母通常是代码总量，分子通常是受变更影响的代码行数。

12. 关于“系统开发项目中的软件质量管理指标”。

### 知识点总结：软件质量管理指标

软件质量管理（Quality Management）不仅仅是看代码写得好不好，更需要**定量的、可测量的数据指标**来评估。

- **质量管理指标的特点**：它们通常关注缺陷、测试覆盖率、评审情况或技术债务，旨在预防错误并提高系统稳定性。
- **软件质量的维度**：包括正确性（Correctness）、可靠性（Reliability）、易维护性（Maintainability）等。

### 选项解析与详细说明

- **ア：WBSを構成するワークパッケージの完了数 (WBS工作包完成数)**
  - **错误**：这是衡量**进度 (Schedule)** 的指标，用于监控项目进展是否按计划完成，与“软件质量”无关。

- **イ：個人別のプログラミングの生産性 (个人编程生产力)**
  - **错误：**这是衡量**效率 (Efficiency)** 或人力绩效的指标，不直接反映软件质量。
- **ウ：成果物ごとのレビュー時間 (每个成果物的评审时间)**
  - **正确：**这是一种常见的**过程质量管理指标**。通过在评审阶段投入充分的时间，可以尽早发现潜在的逻辑错误和缺陷，是保障软件最终质量的关键预防措施。
- **エ：プログラムのバージョン (程序版本)**
  - **错误：**这是**配置管理 (Configuration Management)** 的内容，用于版本追踪，不属于质量管理评价指标。

## 小学生级记忆法

想象你在写一份“满分作文”：

- **进度管理 (选项ア)：**看你写了几段，写没写完。
- **效率管理 (选项イ)：**看你写作文的速度有多快。
- **质量管理 (选项ウ)：**看你写完后花了多少时间认真“检查、润色、修改错别字”。**花在评审上的时间越长，作文里的错误通常就越少，这就是质量指标！**

## 正确答案

**ウ (成果物ごとのレビュー時間)**

## 考试小秘籍

- **抓关键词：**题目问的是“软件质量 (ソフトウェア品質)”。在备考中，凡是看到“质量”，脑子里就要搜索“评审 (レビュー)”、“缺陷密度 (バグ密度)”、“测试覆盖率 (テストカバレッジ)”这些词。
- **排除法：**凡是涉及“时间进度”、“人数”、“个人绩效”的，绝大多数情况下都不属于质量管理指标。

13. 这一道题考查的是 **软件规模估算方法 (ソフトウェア開発の規模見積もり)** 中的 **功能点法 (Function Point Method)**。

## 知识点总结 (ファンクションポイント法)

功能点法是一种不依赖代码行数 (LOC)，而是基于**用户视角的功能需求**来定量估算软件规模的方法。

- **核心思想：**通过识别软件的功能项（输入、输出、查询、内部逻辑文件、外部接口文件），并根据其复杂程度进行加权计算，从而量化软件的规模。
- **适用场景：**适用于需求阶段，能够较好地反映软件的功能复杂度，且具有客观性和可重复性。

| ※複雑さの補正係数 0.75 |    |    |       |
|----------------|----|----|-------|
| ファンクションタイプ     | 個数 | 重み | FP    |
| 外部入力           | 2  | 4  | 6     |
| 外部出力           | 3  | 5  | 11.25 |
| 内部論理ファイル       | 1  | 10 | 7.5   |
| 外部インタフェースファイル  | 1  | 8  | 6     |
| 外部照会           | 2  | 4  | 6     |
| 合計FP           |    |    | 36.75 |

図 ファンクションポイント法

## 选项解析与详细说明

- **ア：(LOC法/标准任务法)**
  - 描述的是基于已知开发规模（如代码行数）进行后续推算的逻辑，不属于功能点法。
- **イ：(类比估算法 / 類推見積法)**
  - 描述的是基于过去类似项目的经验数据进行调整的估算方法，属于类比估算。
- **ウ：(功能点法 / ファンクションポイント法)**
  - **正确。**该选项准确描述了功能点法的核心逻辑：即通过入出力的项目数、文件数进行定量测量，并结合复杂度进行调整来估算规模。
- **エ：(自下而上估算法 / 標準タスク法)**
  - 描述的是将工作分解为单位任务，汇总各任务基准作业量的方法，属于典型的自下而上估算法。

## 小学生级记忆法

想象你在给一家餐厅“估算厨师的工作量”：

- **功能点法：**不管这个厨师切菜动作快慢（不看代码行数），只看他要做的“菜品数量”（**输入输出功能**）以及“菜品的复杂程度”（复杂度调整）。
- 只要菜单列出来，难度定下来，就算出总工作量了。这就是功能点法的逻辑：“**看菜单，估难度，算出量**”。

## 正确答案

ウ

## 考试小秘籍

- **抓关键词：**在考题中，只要看到“入出力データ数”、“ファイル数”、“機能”、“定量的な計測”这些词，答案几乎 100% 就是“ファンクションポイント法”。

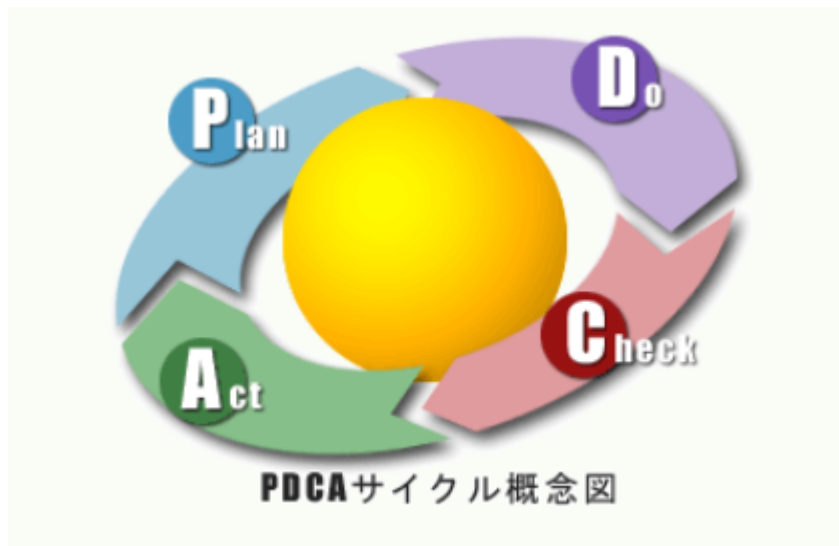
- **思维陷阱**：功能点法最大的特点是“与具体的编程语言和实现方式无关”，因为它衡量的是“功能”，而不是“代码实现”。

14. 这道题考查的是项目管理与质量管理中最核心的管理闭环模型：**PDCA 循环**。

### 💡 知识点总结

PDCA 循环是持续改进流程的基石，无论是在软件开发还是日常管理中都极其常用。它由四个阶段组成：

- **P (Plan)**：计划，设定目标和方案。
- **D (Do)**：执行，按照计划进行实施。
- **C (Check)**：检查，评估实施结果，对比目标与实际偏差。
- **A (Act)**：行动（或处理），根据检查结果进行改进或标准化。



### 🧠 小学生级记忆法

把 PDCA 想象成“减肥计划”：

- **P (Plan)**：你制定了“下个月瘦 5 斤”的计划。
- **D (Do)**：你每天去跑步、少吃饭。
- **C (Check)**：月底你上秤称体重，检查（Check）看是不是真的瘦了。
- **A (Act)**：发现没瘦，于是改进（Act）方案，换一种运动方式。
- 记住：C 就是去秤上“Check”一下！

15. 如题：ウォーターフォール型のソフトウェア開発において、運用テストで発見された誤りの修復に要するコストに関する記述のうち、適切なものはどれか。

本题考查的是ウォーターフォールモデル（瀑布模型）中开发阶段与修复成本之间的核心逻辑。

a. 不可逆性与阶段推移：

b. 在ウォーターフォールモデル中，开发是像瀑布一样自上而下按部就班进行的：

1. 要求定義 → 外部設計 → 内部設計 → コーディング → 各種テスト → 運用テスト

c. 修復コスト（修复成本）的递增规律：

- 发现错误的阶段越晚，修复成本呈指数级暴涨。
- 源头错误代价最高：在上游阶段（如要求定義、外部設計）犯下的错误，如果没有及时发现而一路向下传递，到了最后期（運用テスト）才暴露，那么从设计书、代码到测试用例全部都要推翻重来。
- 下游错误代价较低：在纯编码（コーディング）阶段犯的错，往往只需要改动局部代码并重新测试该模块即可，不会动摇地基。

## 选项解析与详细说明

- ア：外部設計の誤りは、プログラムだけでなく、マニュアルなどにも影響を与えるので、コーディングの誤りに比べて修復コストが高い。
  - 【正解】 外部設計（面向用户的设计）如果错了，不仅影响后期的プログラム（程序），还会连带影响操作手册（マニュアル）等一系列产出物。因为它是上游阶段的错误，波及范围极广，所以修复コスト显著高于纯下游的コーディング（编码）错误。完全符合瀑布模型的逻辑。
- イ：コーディングの誤りは、修復のための作業範囲がその後全工程に及ぶので、要求定義の誤りに比べて修復コストが高い。
  - 【错误】 前后因果与大小关系完全颠倒。コーディング是下游，要求定義是最上游。要求定義如果出错，后续的全体工程全部都要重做，其代价远超コーディング错误。
- ウ：テストケースの誤りは、テストケースの修正とテストのやり直しだけでは済まないの  
で、外部設計の誤りに比べて修復コストが高い。
  - 【错误】 测试用例（テストケース）出错，通常只需要修正测试用例并重新执行测试即可（テストのやり直しだけで済むことが多い）。而外部設計错误会导致程序架构重构，代价显然是外部設計错误更高。
- エ：内部設計の誤りは、設計レビューによってほとんど除去できるので、もし発見されても、コーディングの誤りに比べて修復コストは安い。
  - 【错误】 题目明确指出是在「運用テストで発見された」（在运用测试中才被发现）。虽然通过設計レビュー（设计评审）理论上可以发现很多错误，但在最后期（運用テスト）才被揪出来的内部設計错误，由于已经经过了编码和多次测试，需要倒回去修改设计书、重新写码、重新测试，因此其修复成本绝对比单纯的コーディング错误要高。

## 处理问题的排查表

虽然本题为概念理解题，但针对瀑布模型中“中后期发现错误”的排查与代价评估，我们可以梳理出如下排查思考链条：



| 步骤     | 排查/评估对象         | 核心确认点                         | 结论与行动建议      |
|--------|-----------------|-------------------------------|--------------|
| Step 1 | 确定Bug发生源头       | 这个Bug是在哪个阶段（上游还是下游）被埋下的？      | 如果是设计阶段引入的，警 |
| Step 2 | 评估波及范围 (Impact) | 修正这个Bug需要改动哪些产出物？（设计书？代码？手册？） | 影响的产出物种类越多，修 |
| Step 3 | 追溯当前测试阶段        | 当前处于哪一个テスト阶段？（单体、系统、还是运用？）    | 越接近运用测试（運用テス |

## 小学生级记忆法

### 盖房子的“建材与地基”理论

把开发软件想象成盖一座高档别墅：

- 要求定義 / 外部設計 = 房子的设计图纸（地基在哪里、承重墙在哪里）。
- コーディング = 刷墙漆、贴瓷砖。
- コーディングの誤り：墙漆颜色刷错了。
  - 大管家拍大腿：“没事，就把这面墙重新刷一下就行，花不了几个钱！”（成本低）
- 外部設計の誤り：承重墙位置画错了，房子都盖到封顶了（運用テスト）突然发现地基要塌。
  - 大管家崩溃：“完了！不仅要砸墙，连里面的水管、电线、地毯全废了，得全拆了重来！”（成本极高）

## 秒杀口诀

越上游，越地基；错得越早，赔得越多！ > 写代码，像刷漆；后期发现，也是小伤！

## 正确答案

ア

## 考试小秘籍

- 直击关键词：在基本情报考试中，看到「ウォーターフォール」（瀑布模型）和「修復コスト」（修复成本）连用时，核心逻辑只有一条——“阶段越靠前（上游）的错误，在后期被发现时修复成本一定越高”。
- 机械式排除法：直接比大小。
  - 选项イ：コーディング（下）  $\$>\$$  要求定義（上）  $\$ \rightarrow \$$  直接秒杀排除。
  - 选项ウ：测试用例（下）  $\$>\$$  外部設計（上）  $\$ \rightarrow \$$  直接秒杀排除。
  - 选项エ：内部設計（上）  $\$<\$$  コーディング（下）  $\$ \rightarrow \$$  直接秒杀排除。

- 只有选项**A**正确表达了 外部設計（上）  $\$>\$$  コーディング（下），锁定答案！

16. ある開発プロジェクトの全体の開発工数の見積りは88人月である。作業を開始した1月から5月までは各月10名を投入したが、5月末時点で40人月分の作業しか完了していない。8月末までにこのプロジェクトを完了するためには、6月以降あと何名の追加要員が必要か。ここで、6月以降のすべての要員の作業効率は、5月までの要員と同じであるものとする。

本题考查的是项目管理中的開発工数（开发工时/人月）与作業効率（工作效率）的计算。这是下午题和上午题都极度高频的经典“算账题”。

要攻克这类题，必须彻底理解以下三个核心概念：

a. 人月（にんげつ / ひとつき）：

- 它是衡量工作量的单位。1人月 代表“1个人全职工作1个月”所能完成的工作量。
- 总工时（人月）= 人数 × 月数。

b. 作業効率（工作效率）：

- 计划是理想的，但现实中每个人的产出效率可能打折。
- 实际工作效率 =  $\frac{\text{实际完成的工数（人月）}}{\text{实际投入的总人月}}$

c. 动态调整逻辑：

- 发现进度滞后时，需要用“剩下的工作量”除以“剩下的时间”，并结合“实际工作效率”来重新计算后续需要维持的**总人数**，最后减去原有人员，得到**追加人员（追加要員）**。

## 选项解析与详细说明

我们用严密的逻辑一步步拆解这个项目的现状和未来：

### 第一步：诊断现状（1月～5月末）

- 计划投入：1月到5月共 5个月，每月投入 10名。
  - 实际已经消耗的劳动力 = 10名 × 5个月 = 50人月。
- 实际产出：题目说明，5月末时只完成了 40人月 分的工作。
- 计算实际工作效率：
  - 这帮人出了50人月的力，却只干了40人月的活。
  - 作业效率 =  $\frac{40\text{人月}}{50\text{人月}} = 0.8$  （即每个人只能发挥出正常人 80% 的战斗力）。

### 第二步：规划未来（6月～8月末）

- 剩余工作量：总预算是 88人月，已经干完了 40人月。
  - 剩下还要啃的硬骨头 = 88 - 40 = 48人月。
- 剩余时间：从6月到8月末（6月、7月、8月），总共只有 3个月。

◦ 计算后续每月需要的“标准战力”：

- 要在3个月内干完48人月的活，每个月必须雷打不动地推进：
- $\frac{48\text{人月}}{3\text{个月}} = 16\text{人月/月}$  的工作量。

### 第三步：计算需要的人数与追加人员

- **考虑效率折损（0.8）**：因为每个人的效率只有 0.8，为了达到每月 16人月 的实际产出，每月需要的实际总人数为：
- $\frac{16\text{人月}}{0.8} = 20\text{名}$ 。

**计算追加人数**：现在团队里已经有原有的 10名 员工了。

- 还需要额外拉来多少壮丁 = 20名 - 10名 = 10名。
- **A：6** —— 错误。忽略了员工效率打折（按效率1.0计算就会错算出这个答案）。
- **I：10** —— **【正解】** 完全符合上述逻辑计算。
- **U：16** —— 错误。这是6月以后每月需要的总战力（人月），而不是“追加的人数”。
- **E：20** —— 错误。这是6月以后每月需要的**总人数**，题目问的是**追加（あと何名）**。

### 小学生级记忆法

#### “吃蛋糕”理论

把整个项目想象成吃一个 88斤 的大蛋糕。

#### 上半场（1月~5月）：

- 来了 10 个小朋友，吃了 5个月。
- 妈妈过来一称，发现蛋糕只被吃了 40斤。
- 大管家掐指一算：“不对啊，10个人吃5个月，本该吃50斤的，怎么只吃了40斤？这帮小家伙在摸鱼！每个人只出了 80% 的力（效率 0.8）！”

#### 下半场（6月~8月）：

剩下的蛋糕 = 88斤 - 40斤 = 48斤。

剩下的时间 = 只剩 3个月 了。

也就是说，每个月必须吃掉：48 ÷ 3 = 16斤 蛋糕。

可是小朋友会摸鱼（效率0.8），所以为了按时吃完，每个月座位上必须坐：

16 ÷ 0.8 = 20个人。

大管家大喊：“本来就有10个人了，快，再给我叫 10 个人（追加10名）进来一起吃！”

### 秒杀口诀

算人月，先算效率折几成。

剩多少活，除以时间，再除效率。

算出总人数，别忘减去老员工！

## ✓ 正确答案

イ

## 考试小秘籍

- 防错警惕：看清问的是“总数”还是“追加”。
  - 考试方非常坏，经常会把计算出来的中间结果（比如总人数 20）放在选项 **工** 里诱骗你。看到题目末尾的「あと何名」或者「追加要員」，一定要用 总人数 - 现有人员。

- 机械式公式套用：

看到“工作效率和5月前相同”，直接列式：

$$\text{效率} = \frac{40}{(10 \times 5)} = 0.8$$

$$\text{总人数} = \frac{(88 - 40)}{(3 \times 0.8)} = 20$$

追加 =  $20 - 10 = 10$ 。一分钟内无痛拿分！

17. プロジェクトのリスクに対応する戦略として、損害発生時のリスクに備え、損害賠償保険に加入することにした。PMBOKによれば、該当する戦略はどれか。

本题考查的是 PMBOK（项目管理知识体系）中关于 **マイナスのリスクへの対応戦略（消极风险/威胁的应对策略）**。

当项目中出现潜在的负面风险时，PMBOK 规定了四种最核心的应对手段。在基本情报考试中，这四个术语的定义属于 **必考送分题**：

a. 回避（かいひ / Avoid）：

- 彻底消除风险。变更项目计划，让风险发生的概率直接降为 0。例如：因为天气太冷可能导致系统故障，干脆取消冬天的开发计划。

b. 軽減（けいげん / Mitigate）：

- 降低概率或减轻减轻损害。通过提前采取措施，把风险发生的可能性或它带来的损失变小。例如：为了防止服务器宕机，做个双机备份（不保证不宕机，但宕机了能快速恢复）。

c. 転嫁（てんか / 移転 / Transfer）：

- 甩锅给第三方。把风险带来的财务损失或应对责任转让给外人。买保险、把高风险模块外包（アウトソーシング）是最典型的转嫁。

d. 受容（じゅよう / Accept）：

- **躺平接受。**不采取任何主动措施，等风险真的发生了再说，或者只留一笔备用金（予備費）。



## 选项解析与详细说明

题目中明确提到：「損害賠償保険に加入することにした」（加入了损害赔偿保险）。

- **ア：回避**
  - **【错误】** 买保险并不能阻止“损害发生”这件事。风险依然存在，没有被消灭，所以不是回避。
- **イ：軽減**
  - **【错误】** 保险无法降低事故发生的概率，也无法缩减事故本身的破坏力，它只负责事后赔钱。
- **ウ：受容**
  - **【错误】** 受容是不做积极干预（或仅备预备费）。而买保险是花真金白银主动把风险转出去的行为。
- **エ：転嫁**
  - **【正解】** 买保险的本质就是“交了保费，把一旦出事需要赔偿巨款的经济风险，转嫁给保险公司承担”。完美符合「转嫁」的定义。



## 处理问题的排查表

当项目经理（PM）识别到一个负面风险时，决定采用哪种策略的思考排查模型：

| 风险特征       | 核心排查考量           | 对应最佳战略 | 经典考题关键词          |
|------------|------------------|--------|------------------|
| 致命且无法承受    | 能不能直接改计划绕过去？     | 回避     | 变更计划、中止、采用成熟技术   |
| 可控但有概率发生   | 能不能通过提前做准备来减小损失？ | 軽減     | 备份、测试、教育训练、冗余设计  |
| 技术不足/财务风险高 | 能不能花钱让别人替我扛这个风险？ | 转嫁（転嫁） | 保険に加入、外包（委託）     |
| 影响极小/无法避免  | 采取措施的成本是不是比损失还高？ | 受容     | 保持现状、准备对策资金（予備費） |



## 小学生级记忆法



### “新手司机开车上路”理论

把项目风险想象成新手司机开车出门，担心发生撞车事故：

- **回避：**“外面下大暴雪，我干脆不出门、不开车了！”（风险直接归零）
- **軽減：**“出门前，我把安全带系好、车速开慢点。”（就算撞了，伤得也轻点）
- **转嫁：**“我花钱买了个全险，一旦撞了车，保险公司全额赔付！”（把要赔的钱甩锅给保险公司）

- 受容：“撞就撞吧，反正我这破车就值500块，撞烂了直接扔掉换新的。”（躺平接受）

## 秒杀口诀

买保险、找外包，全是转嫁（転嫁）的大招！改计划、直接取消，那是回避（回避）在求饶！

## 正确答案

エ

## 考试小秘籍

- 机械式关键词绑定：
  - 在日本IT考试中，看到「保险（保険）」或者「外包（他社への委託 / アウトソーシング）」→ 100% 对应「转嫁（転嫁）」。
  - 看到「采用成熟技术（実績のある技術）」、「增加测试（テストの強化）」→ 100% 对应「轻减」。
  - 看到「中止」、「变更计划（計画変更）」→ 100% 对应「回避」。本题看到“保险”二字，直接盲选 **エ**，两秒钟锁定分数！

### 18. システム開発における工数の見積りに関する記述のうち、適切なものはどれか。

本题考查的是系统开发中关于工数見積り（工时估算）的各种经典方法及其核心特征。国家考试中最爱对比考查的就是 **COCOMO** 和 **ファンクションポイント法（Function Point, FP法）**。

#### a. COCOMO（Constructive Cost Model）：

- **核心逻辑**：基于代码行数（プログラムステップ数/ソースコード行数）来估算开发工期和成本的模型。
- **致命特征**：它是由 Barry Boehm 提出的一种参数化模型。其公式中包含很多特定的**系数**（参数）。由于每个公司的技术栈、开发人员水平不同，**必须收集自社（自己公司）过去的生产性数据来调整这些固有系数**，否则估算结果会彻底失真。

#### b. ファンクションポイント法（FP法）：

- **核心逻辑**：基于软件的功能数量和复杂程度（如输入画面数、输出报表数、内部文件数等）来估算规模。
- **最大优点**：**不需要知道代码行数（プログラムステップ数）**。即使在还没有写代码的最上游阶段（用户要求定义阶段），只要定下了要做几个画面和功能，就能把工数估算出来。

#### c. 工数見積りの目的：

- 估算不仅用于进度的控制（進捗管理），它决定了项目规模。而规模直接关系到测试用例的密度、Bug密度的基准设定，因此它与**品质管理（品質管理）息息相关**。

## 选项解析与详细说明

- **ア：COCOMO の使用には、自社における生産性のデータ収集が不可欠である。**
  - **【正解】** 如知识点所述，COCOMO 模型依赖特定的经验参数。要想在实际项目中使用它，**必须不可欠（不可或缺）地收集自社过去的开发数据去修正模型参数，使其符合本公司的实际生产力。**
- **イ：開発要員の技量が異なるので、過去に開発したプログラムの規模は見積りの参考になるが、工数は参考にならない。**
  - **【错误】** 虽然人员水平有差异，但过去类似项目的“实际耗费工数（工数）”依然是极其重要的参考基准（类比估算法/類推見積法）。绝对不能说“工数は参考にならない”。
- **ウ：工数の見積りは、作業の進捗管理に有効であるが、ソフトウェアの品質管理には関係しない。**
  - **【错误】** 工数和规模估算决定了后续测试阶段应该投入多少资源、抓出多少 Bug。在 IT 行业中，常用「バグ密度」（每千行代码或每人月应该有多少 Bug）来衡量质量。因此它与**品质管理**密切相关。
- **エ：ファンクションポイント法による見積りでは、プログラムステップ数を把握する必要がある。**
  - **【错误】** 斩断因果！FP法（ファンクションポイント法）的核心价值恰恰在于“不需要（不用把握）プログラムステップ数（代码行数）”就能估算出结果。需要代码行数的是COCOMO 法。



## 处理问题的排查表

当我们在考试或实际现场中，需要根据项目特征选择“估算方法”时的思考排查模型：

| 团队/项目现状     | 核心关注点/限制条件                      | 适合的估算方法           | 考试秒杀关键词               |
|-------------|---------------------------------|-------------------|-----------------------|
| 处于开发最早期     | 代码一行都没写，只知道要做出几个画面、几个数据库表格。     | ファンクションポイント法（FP法） | 機能の数、画面数、帳票数、上流工程     |
| 公司积累了大量历史代码 | 能够大致预测出系统总共需要写几万行代码，且有公司专属的系数库。 | COCOMO            | ソースコード行数、固有の係数、自社のデータ |
| 时间紧急/有类似旧项目 | 没有任何理论公式，全凭历史经验或专家拍脑袋。          | 類推見積法（デルファイ法など）   | 過去の類似例、有識者の経験         |



## 小学生级记忆法



## “裁缝量体裁衣”理论

把软件开发估算工数，想象成裁缝店要给客户做衣服，估算需要多少工时和布料：

- **COCOMO = “数针脚法”**
  - 这个方法认为：衣服有多大，取决于要缝多少针（代码行数）。



- 大管家摇头：“每个裁缝手速不一样，胖子裁缝和瘦子裁缝1分钟缝的针数完全不同！你想用这个公式，必须先在自己店里测试并收集每个裁缝的缝纫速度数据（自社のデータ収集が不可欠），不然公式根本不准！”
- FP法（ファンクションポイント法）= “数口袋法”
  - 这个方法不看针脚，只看衣服的功能：这件衣服有2个袖子、4个口袋、1条拉链（画面数、功能数）。
  - 大管家拍手：“太好了！我根本不需要知道你要缝多少针（不需要プログラムステップ数），只要数数口袋数量，我就能把工时和价格算出来！”

## 秒杀口诀

COCOMO 靠行数，自社数据来调参数。

FP 算功能，不要代码也能成！

## 考试小秘籍

- 机械式错位排除法：
  - 在基本情报考试中，只要看到「ファンクションポイント法」后面连着「プログラムステップ数（代码行数）が必要」→ 100% 属于张冠李戴，直接排除（如选项工）。
  - 只要看到任何管理工具说「〇〇管理には関係しない」（与某管理无关）这种绝对化的消极表述 → 99% 都是错的，直接排除（如选项ウ）。项目管理中万物互联，工时、进度、质量永远相互影响。

19. システム開発において、工数(人月)と期間(月)の関係が次の近似式で示されるとき、工数が4,096人月のときの期間は何か月か。

$$\text{期間} = 2.5 \times \text{工数}^{1/3}$$

本题表面上是一个项目管理题，但本质上考查的是数学中的指数与立方根（三乘根）计算。这也是COCOMO 模型在实际演算法中的一种简化数学映射。

### a. 分数的指数含义：

当指数为分数时，诸如  $X^{1/3}$  这样的表达式，其数学含义就是求  $X$  的三乘根（开三次方）。

$$\text{工数}^{1/3} = \sqrt[3]{\text{工数}}$$

也就是说，我们要寻找一个数字，让它自己连续相乘3次之后恰好等于这个工数。

### b. 公式代入与变形：

题目给出了近似公式：

$$\text{期間} = 2.5 \times \text{工数}^{1/3}$$

已知 工数 = 4,096，我们只需要安全、准确地解出  $4096^{1/3}$  是多少，即可轻松破题。

## 选项解析与详细说明

我们来一步步拆解硬核的数学计算逻辑，彻底看清为什么其他选项是干扰项：

### 第一步：求解 $4,096^{1/3}$ （找谁的三次方是 4096）

面对 4,096 这么大的数字，直接去硬猜开三次方会很痛苦。考试时我们可以用**2的幂次方常识**或者**尾数估算法**来降维打击：

- 方法A（IT人的2的幂次直觉）：

- 在计算机科学中，我们对2的次方非常敏感：

$$2^{10} = 1024$$

$$2^{11} = 2048$$

$$2^{12} = 4096$$

- 所以，工数  $= 4096 = 2^{12}$ 。

- 那么，

- $4096^{1/3} = (2^{12})^{1/3} = 2^{12 \times \frac{1}{3}} = 2^4 = 16$

- 方法B（尾数估算法）：

- 如果一时间想不到  $2^{12}$ ，我们可以观察 4096 的末尾数字是 6。

- 任何数连续相乘3次，只有**末尾是6的数**，其次方的尾数才可能是6（因为  $6 \times 6 \times 6 = 216$ ）。

- 我们可以试一下 16：

$$16 \times 16 = 256$$

$$256 \times 16 = 4096。刚好对上！$$

### 第二步：带入公式求期间

既然解出了 工数 $^{1/3} = 16$ ，那么：

$$\text{期间} = 2.5 \times 16$$

为了口算不出错，我们可以把 2.5 拆成  $\frac{5}{2}$ ：

$$\text{期间} = \frac{5}{2} \times 16 = 5 \times 8 = 40 \text{个月}$$

- ア：16** —— 错误。这是  $4096^{1/3}$  计算出来的中间结果，考试方故意把它放第一个来当做“诱饵陷阱”。
- イ：40** —— **【正解】** 完美符合计算结果。
- ウ：64** —— 错误。这是 4096 开平方根（ $\sqrt{4096} = 64$ ）的结果，属于看错指数的算错方向。

- **工：160** —— 错误。属于在计算  $2.5 \times 16$  时把小数点点错位置的低级失误

## 🧠 小学生级记忆法

### 📦 “俄罗斯方块大礼盒” 理论

把 4,096 想象成一个巨大的正方体大礼盒，里面密密麻麻装了 4,096 个小正方形方块。

- **工数<sup>1/3</sup>** 的含义：就是让你找出这个大礼盒的“边长”是多少个方块。
- **大管家开始挑边长：**

如果边长是10，那长 × 宽 × 高就是  $10 \times 10 \times 10 = 1000$  个（太小了！）。

如果边长是20，那长 × 宽 × 高就是  $20 \times 20 \times 20 = 8000$  个（太大了！）。

既然大礼盒装了 4,096 个，说明边长一定在 10 到 20 之间，而且最后一位相乘要等于6。

大管家一拍大腿：“试一下 16！ $16 \times 16 \times 16$  刚好等于 4096！边长找到了，就是 16！”

找到了边长 16，别忘了妈妈的公式还要让你乘上 2.5 倍的服务费：

$$16 \times 2.5 = 40 \text{ 块钱!}$$

## 🧠 秒杀口诀

看到指数三分之一，就是找谁的三次方。

4096 别慌张，它是 2 的 12 次方。

开完立方等于 16，乘上系数才是王道！

## ✅ 正确答案

イ

## 📝 考试小秘籍

- **系数乘法的小技巧（2.5倍速算）：**

在考场上算「乘以 2.5」时，用“先翻倍，再加一半”的方法最不容易错。

16 的 2 倍是 32。

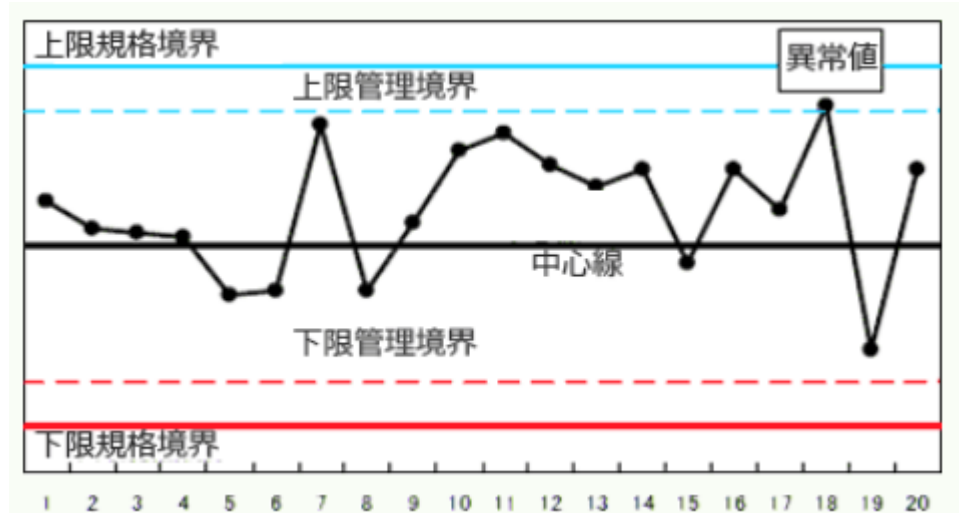
16 的一半（0.5 倍）是 8。

$32 + 8 = 40$ 。口算只需一秒，而且绝对不会点错小数点！

- **机械式排除陷阱：**

- 记住，**算出来的第一个数字往往不是答案**！当你千辛万苦算出 16 时，发现选项 ア 就是 16，这时候一定要掐自己一下：公式前面的 2.5 我乘了吗？只要保持这个警惕，这种题就是纯送分！

- a. 生産物の品質を時系列に表し, 生産工程が管理限界内で安定した状態にあるかどうかを判断するための図は**管理図**



### 基本定义与目的:

- 管理図是一种将产品的特性值按时系列（时间顺序）进行打点连接的折线图。
- 它的核心目的是：**判断制造工程是否处于稳定状态（工程が安定した状態にあるか）**，从而提前发现异常，防止不良品流入下个环节。

### 三大核心线（基于统计学计算，通常是 $\pm 3\sigma$ ）：

- 中心線 (CL / Central Line)：**通常是平均值，代表理想的稳定中心。
- 上方管理限界線 (UCL / Upper Control Limit)：**图中写作“上限管理境界”，是工程允许波动的统计学上限。
- 下方管理限界線 (LCL / Lower Control Limit)：**图中写作“下限管理境界”，是工程允许波动的统计学下限。

### 异常判定逻辑：

- 超出界限即异常：**只要打点**超出了管理限界线**（如图片中第18天的异常值），就说明工程发生了非偶然导致的“**异常原因**”，必须立刻停线排查。
- 注意辨析：**管理限界线（统计出来的控制范围）和规格境界线（客户要求的公差范围）是两个概念。考试中通常重点关注点是否掉出**管理限界线**。



## 处理问题的排查表

当我们在工厂现场或考试题目中，看到管理图上的点出现异常时，标准的排查思考链条如下：

| 现象（管理図の状態）           | 状态判定      | 潜在根本原因排查                     | 现场PM/工程师的行动                 |
|----------------------|-----------|------------------------------|-----------------------------|
| 点全部在控制线内，且无规律随机分布    | 工程安定（正常）  | 属于正常的自然偶然波动。                 | 无需干预，保持当前状态。                |
| 有单点蹦到了上限/下限管理限界线之外   | 工程异常（突发性） | 换了新工人、机器刀具突然断裂、原材料突然变质。      | <b>立刻停线！</b> 彻查该日发生了什么变更。   |
| 点都在线内，但连续7点以上不断上升/下降 | 工程异常（趋势性） | 机器轴承在逐渐磨损、刀具慢慢变钝、药液浓度逐渐蒸发变浓。 | 虽然还没超线，但已经具有趋势性，必须在超线前进行干预。 |

### 陷阱陷阱一：时系列 vs 因果关系（混淆管理图与散布图）

- 真题错误选项常用套路：「品質の因果關係を表すために管理図を用いる」（为了表示质量的因果关系而使用控制图）。
- 彻底拆解：管理图看的是时间推移（时系列）下的稳定性。看“因果关系（两个变量的相关性）”的那叫散布图（さんぶず）。

### 陷阱二：目的混淆（混淆管理图与パレート図）

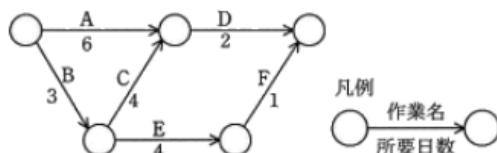
- 真题错误选项常用套路：「不良品の件数が多い順に並べて、重点管理項目を見つけるために管理図を用いる」（为了按不良品数量从大到小排列、找出重点管理项目而使用控制图）。
- 彻底拆解：按大小排列、用于抓主要矛盾（二八定律）的是パレート図（排列图）。管理图的唯一目标是看工程稳不稳定、有没有突变。

### 陷阱三：管理限界线 vs 规格线（規格値）

- 真题错误选项常用套路：「上限管理限界線は、顧客が指定した製品の公差（規格）に基づいて引かれる」（上限控制线是基于客户指定的公差引出的）。
- 彻底拆解：大错特错！管理限界线是基于现场实际生产数据的统计学计算（一般是  $\mu \pm 3\sigma$ ）得出来的，反映的是生产线的“自身实力”；而规格线（图中红线）才是客户给的“及格线”。

## 20. 如题

図のアローダイアグラムにおいて、プロジェクト全体の期間を短縮するために、作業A～Eの幾つかを1日ずつ短縮する。プロジェクト全体を2日短縮できる作業の組合せはどれか。



### 多路径的动态竞争关系（陷阱所在！）：

- 当你把最长路径缩短后，原本第二长的路径可能会突然逆袭，变成新的关键路径。
- 此时，如果想让总工期继续缩短，就必须同时缩短所有并列最长的关键路径。

先，我们必须把图中**所有的完整路径**全部列出来，算出它们各自的总工期（由于题目只问作业 A～E，最后的 F（1天）是公共必经之路，我们直接代入完整计算）：

**路径①：**  $A \rightarrow D \rightarrow \text{终点}$  （上方路线）

- 总工期 =  $6 + 2 = 8$ 天

**路径②：**  $B \rightarrow C \rightarrow D \rightarrow \text{终点}$  （中间路线）

- 总工期 =  $3 + 4 + 2 = 9$ 天

**路径③：**  $B \rightarrow E \rightarrow F \rightarrow \text{终点}$  （下方路线）

- 总工期 =  $3 + 4 + 1 = 8$ 天

【现状诊断】：当前项目的关键路径是 **路径②**，总工期为 9天。

题目要求：**整个项目总工期要缩短 2 天（也就是要把 9天 变成 7天）**。我们对备选项（真题通常为 A～E 组合，这里我们直接解出正确组合的逻辑）逐一进行沙盘推演：

**核心推演：如何安全减去 2 天？**

- **尝试 1：只缩短当前的霸主（路径②）**

■

路径② 包含  $B, C, D$ 。如果我们把中间最显眼的  $C$  **缩短 2 天**（4天 → 2天）：

- 路径② 变为了： $3 + 2 + 2 = 7$ 天。
- **但是！** 此时路径① 依然是 8天，路径③ 依然是 8天。
- 结果：整个项目的总工期被卡在了 8天，实际只缩短了 1 天。**失败！**

- **明确破局点：必须同时压制三条路径**

- 目标是让 路径①（8天）、路径②（9天）、路径③（8天）全部降到 7天 或以下。
- 我们观察三条路径的作业交集：
  - $B$  是 路径② 和 路径③ 的公共交集。
  - $D$  是 路径① 和 路径② 的公共交集。

- **终极正确组合方案：将  $B$  缩短 1 天，同时将  $D$  缩短 1 天。**

- 让我们来看这时候的奇迹变化：

- 路径①（含  $D$ ）： $6 + (2 - 1) = 7$ 天。
- 路径②（含  $B$  和  $D$ ）： $(3 - 1) + 4 + (2 - 1) = 5 + 2 = 7$ 天。
- 路径③（含  $B$ ）： $(3 - 1) + 4 + 1 = 7$ 天。

三条路线同时完美卡死在 7天！整个项目成功缩短了 2天（9天 → 7天）。

（注：真题中对应的正确选项即为包含「 $B$  与  $D$ 」的选项。）

## 小学生级记忆法

### “三辆赛车同时抵达”理论

- 聪明的办法：

找个1号车和2号车都要经过的收费站  $D$ ，让它通关快1分钟。

再找个2号车和3号车都要经过的红绿灯  $B$ ，让它通关快1分钟。

- 2号车享受了双重加速（减了2分钟），1号和3号各享受了一次加速（各减了1分钟）。大家都拿到了7分钟的好成绩！

把这个项目想象成三辆赛车（路径①、②、③）在不同的车道上比赛，必须等最后一辆车到了才算结束。

- 现状：

1号车（路径①）花了8分钟。

2号车（路径②）花了9分钟。

3号车（路径③）花了8分钟。

- 大管家掐表：“现在最后一名是2号车，总成绩是9分钟。”

- 教练要求：让大家在7分钟内全部到齐。

- 2号车享受了双重加速（减了2分钟），1号和3号各享受了一次加速（各减了1分钟）。大家都拿到了7分钟的好成绩！

### 考试小秘籍

- 考场选项代入大法（终极降维打击）：

- 如果你一时间找不到  $SB$  和  $SD$  的逻辑，最简单、绝对不会错的方法就是“代入法”！
- 直接看题目给的ア、イ、ウ、エ四个选项。比如选项说是“A和C各减1天”，你就直接在图上把A涂改成5，C涂改成3，然后重新算一遍三条路的总长，看最长的那条是不是变成了7。
- 只要代入到包含B和D的选项时，你会发现三条路径全变成了7，高下立判，直接拿分！

21. 本题考查的是利用 **ファンクションポイント法（FP法）** 计算项目的**全工数（总人月）**。这属于典型的“层层套娃题”，考纲核心逻辑在于对项目不同部分工数的**归类与百分比计算**。

ある新規システムの機能規模を見積もったところ、500FP(ファンクションポイント)であった。このシステムを構築するプロジェクトには、開発工数のほかに、システム導入と開発者教育の工数が、合計で10人月必要である。また、プロジェクト管理に、開発と導入・教育を合わせた工数の10%を要する。このプロジェクトに要する全工数は何人月か。ここで、開発の生産性は1人月当たり10FPとする。

**開発工数（纯开发工时）的换算：**



- 纯开发工数（人月） =  $\frac{\text{系统总功能规模（FP）}}{\text{开发生产性（FP/人月）}}$

### 总工数的叠加逻辑：

- 项目的总工数（全工数）通常由三个核心板块相加组成：  
全工数 = 纯开发工数 + 相关辅助工数（导入、教育等） +  
◦ プロジェクト管理工数（管理工时）

### 比例基准的圈定（陷阱所在！）：

- 必须看清管理工数的“10%”到底是以谁为基准。本题明确指出是「開発と導入・教育を合わせた工数の10%」（开发、导入与教育相加之后的总和的10%）。

我们像剥洋葱一样，把这道题的三个工数层层解开：

### 第一层：算出“纯开发工数”

- 题目已知：系统规模为 500 FP，开发生产性为 10 FP/人月（即一个人一个月能干10个点）。
- 那么，纯开发需要多少人月：
- 開発工数 =  $\frac{500 \text{ FP}}{10 \text{ FP/人月}} = 50 \text{ 人月}$

### 第二层：算出“开发 + 导入 + 教育”的合计数

- 题目已知：系统导入和开发者教育的工数合计为 10人月。
- 把开发和这两个辅助工数打包相加：
- 合计数 = 50人月（开发） + 10人月（导入教育） = 60人月

### 第三层：算出“项目管理工数”并求全工数

- 题目已知：プロジェクト管理（项目管理）需要上述合计数（60人月）的 10%。
- 管理工数 = 60人月 × 10% = 6人月
- 最终合体！整个项目需要的**全工数**为：
- 全工数 = 60人月（合计数） + 6人月（管理） = 66人月
- **A：51** —— 错误。这是用 50 人月直接加上了 10 的 10%（1 人月），完全漏掉了导入教育本身的 10 人月。
- **B：60** —— 错误。这是“开发+导入+教育”的中间结果（60 人月），考试方故意放这里诱骗漏算管理工数的考生。
- **C：65** —— 错误。计算错位导致的无意义干扰项。
- **D：66** —— **【正解】** 完美符合上述三层剥洋葱式的严密计算。

## 小学生级记忆法

### “巨无霸汉堡大套餐”理论

把计算全工数想象成去麦当劳点一个巨无霸大套餐：

- **第1步：点汉堡（纯开发）：**

你要吃 500 口肉（500 FP），你每分钟能吃 10 口（生产性 10）。

大管家掐表：“吃完汉堡需要 50 分钟！”

- **第2步：加薯条和可乐（导入与教育）：**

套餐里自带的薯条和可乐，规定必须花 10 分钟 吃完。

大管家合计：“现在主食和配菜一共要花：50 + 10 = 60 分钟！”

- **第3步：付服务费（项目管理）：**

结账时老板说：“我们店要收服务费，服务费是刚才汉堡加配菜总时间的 10%！”

大管家一算：服务费时间 = 60 分钟 × 10% = 6 分钟。

最后你在店里总共坐了多久（全工数）？

汉堡配菜（60 分钟）+ 服务费（6 分钟）= 66 分钟！

### 考试小秘籍

- **机械式一式秒杀（1.1 倍速算法）：**

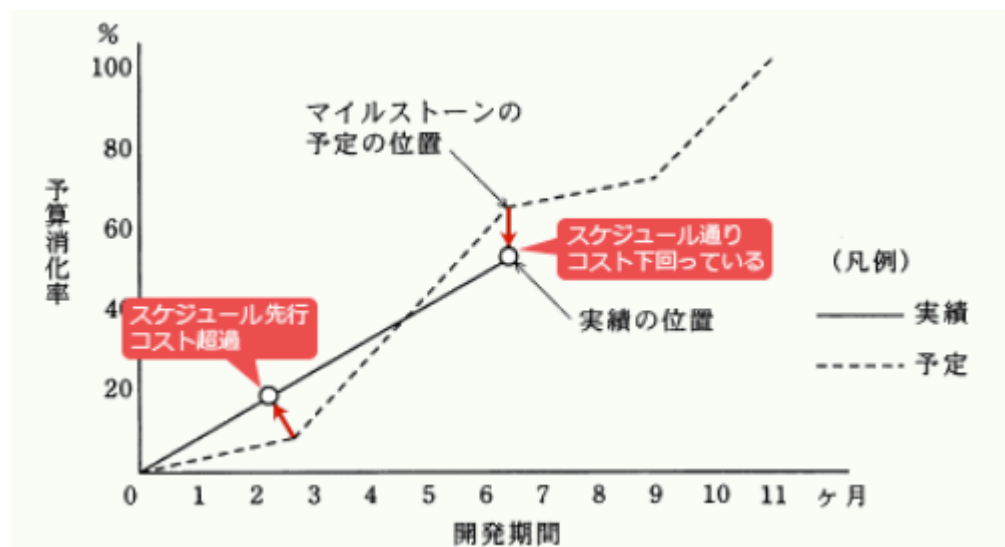
- 凡是看到“某某管理费占前者的 10%”，在数学上就等于求“前者的 1.1 倍”。

- 考场最快列式法： $(500 \div 10 + 10) \times 1.1 = 60 \times 1.1 = 66$ 。一步到位，绝不拖泥带水！

- **“中间答案”陷阱重现：**

- 再次强调，基本情报考试极其喜欢把计算的中间结果（比如 60）放在选项 **イ** 里。看到答案先别激动，回头看一眼题目最后三个字——「全工数」。既然是全工数，就必须把最后的管理费加进去，果断选 **工** 绝不上当！

22. 本题考查的是项目进度与成本管理中极其高频的图表工具——トレンドチャート（Trend Chart / 趋势图）。



### トレンドチャート（趋势图）：

- 核心逻辑：将“作业进度的达成状况”（纵轴通常代表预定或实际完成的条数/百分比）与“预算/费用的消耗状况”（横轴或另一条线代表时间/成本消耗）联系在一起，通过两条折线（折れ線）的交错和走向，来直观预测项目未来会不会超支或延期。

### ガントチャート（Gantt Chart / 甘特图）：

- 核心逻辑：用横向的横条/带状（带状・バー）来表示每个作业的计划日程与实际进度对比。一眼看清“什么时候该干什么”。

### アローダイアグラム（PERT / 箭线图）：

- 核心逻辑：用圆圈和箭头组成网络状（网络状），用来明确表示复杂作业之间的先后顺序和相互依赖关系，并用来计算关键路径。

### RAM（Responsibility Assignment Matrix / 责任分配矩阵，如RACI表）：

- 核心逻辑：用矩阵状（マトリックス状）把“工作任务”和“项目成员”连起来，明确谁负责、谁配合。

## 选项解析与详细说明

- ア：作業に関与する人と責任をマトリックス状に示したもの
  - 【错误】这是 RAM（責任分担マトリックス）或 RACI 表的定义。用于明确组织分工，不属于进度趋势图。
- イ：作業日程の計画と実績を対比できるように带状に示したもの
  - 【错误】看到「带状（带状/条状）」就可以机械式一秒锁定这叫「ガントチャート（甘特图）」。
- ウ：作業の進捗状況と、予算の消費状況を関連付けて折れ線で示したもの
  - 【正解】完美符合トレンドチャート 的标准定义。它通过结合“进度（進捗）”与“预算（予算）”这两条折线，用来监控项目健康度。

- **エ：作業の順序や相互関係をネットワーク状に示したもの**
  - **【错误】** 看到「ネットワーク状（网络状）」就可以机械式一秒锁定这叫「アローダイアグラム（箭线图）」（咱们上一题刚做过它）。

 **处理问题的排查表**

当我们在考场上需要快速识别或现场选用“管理图表”时的排查逻辑如下：

| 画面/描述关键词         | 对应的核心管理工具 (日语原件) | 图形核心外观特征  | PM的核心使用目的        |
|------------------|------------------|-----------|------------------|
| マトリックス状 (矩阵)     | RAM / 責任分担マトリックス | 表格、矩阵交叉点  | 明确“这事该归谁管”。      |
| 带状 (带状) / バー (条) | ガントチャート          | 横向的彩条、进度条 | 宏观对比“计划与实际日程”。   |
| 折れ線 (折线) / 進捗と予算 | トレンドチャート         | 两条波动的折线趋势 | 关联预测“项目会不会人财两空”。 |
| 网络状 (网络状) / 矢印   | アローダイアグラム        | 圆圈加带数字的箭头 | 计算关键路径与找出前后依存关系。 |

 **小学生级记忆法**

 **“减肥与私教的账本”理论**

把管理项目开发，想象成**你在健身房请了私教进行“极限减肥”**：

- **进度（進捗状況）**：你实际上掉了几斤肉（作业完成了多少）。
- **预算（予算の消費）**：你实际上花出去了多少私教费、吃了多少减脂餐（成本消耗）。

教练拿出一张折线图（トレンドチャート）：纵轴画着你的体重下降折线，横轴画着你银行卡里的余额下降折线。

- **大管家一抓头发**：“不好了！你看这两条折线（折れ線）的趋势——钱都要花光了（预算爆表），你身上的肥肉竟然一斤都没少（进度滞后）！再这么发展下去，我们要破产了！”这种把进度和钱包结合在一起看的折线趋势图，就叫トレンドチャート。

 **秒杀口诀**

带状（带状）甘特看日程，网络（ネット）箭线算工期。矩阵（マトリックス）分工抓责任，折线（折れ線）趋势（トレンド）连预算！

 **考试小秘籍**

- 考场纯机械式“关键词一剑封喉”法：在日本 IT 考试的上午题中，关于这四个图表的考题 100% 带有以下固定搭配，直接连连看就能拿分：
  - 「マトリックス」→ 选含有“责任”的选项。
  - 「带状」→ 秒选「ガントチャート」。
  - 「ネットワーク」→ 秒选「アローダイアグラム」。
  - 「折れ線」或「進捗と予算（コスト）」→ 秒选「トレンドチャート」。本题题干问的是「トレンドチャート」，我们在选项中疯狂扫描，瞬间在 **ウ** 看到「予算」和「折れ線」，连题目都不用读完，直接涂黑乌鸦圈，轻松拿下！

23. ステークホルダ(Stakeholder)は、日本語では利害関係者といい、プロジェクトメンバー、プロジェクトマネージャ、顧客、株主、経営者、地域などのように、プロジェクト活動によって利害が生じる可能性のある全ての者です。そのプロジェクトに直接的にかかわる者だけではなく、間接的に影響を受ける可能性のある者も含まれます。また、そのプロジェクトが利益になる者、損害になる者の両方が含まれます。

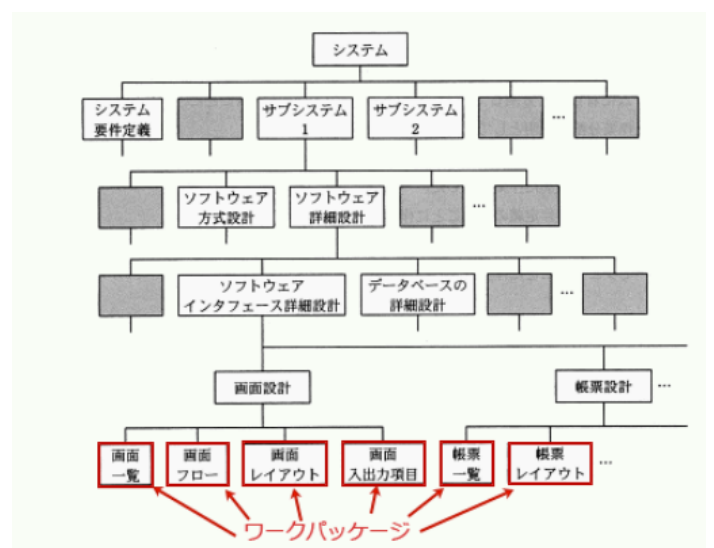
**ステークホルダには、そのプロジェクトが利益になる者、損害になる者の両方が含まれます。**

24. プロジェクトの日程計画を作成するのに適した技法は**PERT**

PERT(Program Evaluation and Review Technique)は、対象となるプロジェクトを完了するために必要となる各工程の依存関係や関連を整理し、分析することで、プロジェクト完了に要する最少日数を特定するための技法です。プロジェクトのスケジュール管理に用いられます。矢印を使ったネットワークで各工程の関連を視覚化した図をPERT図 (別名:アローダイアグラム) といいます。

25. プロジェクトマネジメントで使用するWBSで定義するものは**"プロジェクトで行う作業を階層的に要素分解したワークパッケージ"**

**WBS(Work Breakdown Structure)**は、プロジェクト目標を達成し、必要な成果物を過不足なく作成するために、プロジェクトチームが実行すべき作業を、成果物を主体に階層的に要素分解したものです。この階層的に細分化された構造図の中で最下層に位置する個々の部分をワークパッケージといい、プロジェクトを実施する際のコントロール単位となります。



26. 看图

10人のメンバーで構成されているプロジェクトチームにメンバー2人を増員する。次の条件でメンバー同士が打合せを行う場合、打合せの回数は何回増えるか。

〔条件〕

- 打合せは1対1で行う。
- 各メンバーが、他の全てのメンバーと1回ずつ打合せを行う。

本题考查的是项目管理中的**コミュニケーション経路**（沟通路径/开会组合数）的经典计算。在开发团队中，随着人员的增加，沟通成本和开会次数并不是线性增加的，而是呈指数级暴涨。

a. 组合数公式的运用：

b. 如果一个团队有  $n$  个人，每个人都要和其他所有人进行“1对1”的打交道，那么总共需要的沟通路径（打合の回数）就是从  $n$  个人中任意选出2个人的组合数：

c. 沟通路径数  $= \frac{n \times (n - 1)}{2}$

d. 增量思维（防错核心）：

- 题目要求的不是问“增加后总共开几次会”，而是问「**打合せの回数は何回増えるか**」（开会次数增加了多少回）。
- **增量 = 改变后的总次数 - 原有的总次数。**

## 选项解析与详细说明

我们来严密地分步拆解这场增员前后的开会账单：

### 第一步：算出“增员前”的开会次数（10人状态）

- 初始成员  $n_1 = 10$  人。
- 此时需要的开会总次数为：
- 初始次数  $= \frac{10 \times (10 - 1)}{2} = \frac{10 \times 9}{2} = 45$ 回

### 第二步：算出“增员后”的开会次数（12人状态）

- 增加了2人，团队总人数变为  $n_2 = 10 + 2 = 12$  人。
- 此时需要的开会总次数为：
- 增员后次数  $= \frac{12 \times (12 - 1)}{2} = \frac{12 \times 11}{2} = 66$ 回

### 第三步：计算开会次数的“净增量”

- 题目明确索要的是“增加了多少回”：
- 净增量  $= 66\text{回} - 45\text{回} = 21\text{回}$

- **A: 12** —— 错误。这是误以为新来的2个人只需要每个人和原有的10人各开一次会（ $2 \times 10 = 20$ ，但完全对不上），或者是漏掉了新来的人彼此之间也需要开会的粗心计算。
- **I: 21** —— **【正解】** 完美符合  $66 - 45 = 21$  的增量逻辑。
- **U: 22** —— 干扰项。考试方故意设置的临近数字陷阱，引诱算错乘除法的考生。
- **E: 42** —— 错误。如果没有用组合数公式，而是单纯用  $12 \times 11 - 10 \times 9 = 132 - 90 = 42$ ，就会犯下“忘记除以2”（把A跟B开会、B跟A开会重复算了两次）的经典错误。

## 小学生级记忆法

### “小朋友握手”理论

把1对1开会（打合せ），想象成小朋友们见面拉手手：

- **上半场：房间里有 10 个小朋友。**

老师让他们每个人都互相握一次手。

握手总次数公式： $10 \times 9 \div 2 = 45$ 次。

- **下半场：门外又进来了 2 个新小朋友（小明和小红）。**

老师问：“为了让大家都互相认识，现在需要额外再握几次手？”

大管家开始点名新人的社交任务：

- 新来的小明，必须和原本的 10 个老朋友各握手一次 → **10次**。
- 新来的小红，也必须和原本的 10 个老朋友各握手一次 → **10次**。
- 别忘了！** 小明和小红这两个新来的人，自己彼此之间也得握手一次 → **1次**。

账本合体：整个房间里额外增加的握手次数 =  $10 + 10 + 1 = 21$ 次！

## 秒杀口诀

新人社交分两步，先跟老人全部熟。

新乘以老是基础，新人之间别漏数！

(快捷计算： $2 \text{个新人} \times 10 \text{个老人} + \frac{2 \times 1}{2} = 20 + 1 = 21$ )

## 考试小秘籍

- **无需算两次总数的“增量速算法”：**

如果不想算  $66 - 45$ ，可以直接用上面小学生记忆法里的“新人视角法”：

增量 = (新来人数 × 原有老人数) +  $\frac{\text{新来人数} \times (\text{新来人数} - 1)}{2}$

带入本题： $2 \times 10 + \frac{2 \times 1}{2} = 20 + 1 = 21$ 。只需一步，完全避开了算大数字的麻烦！

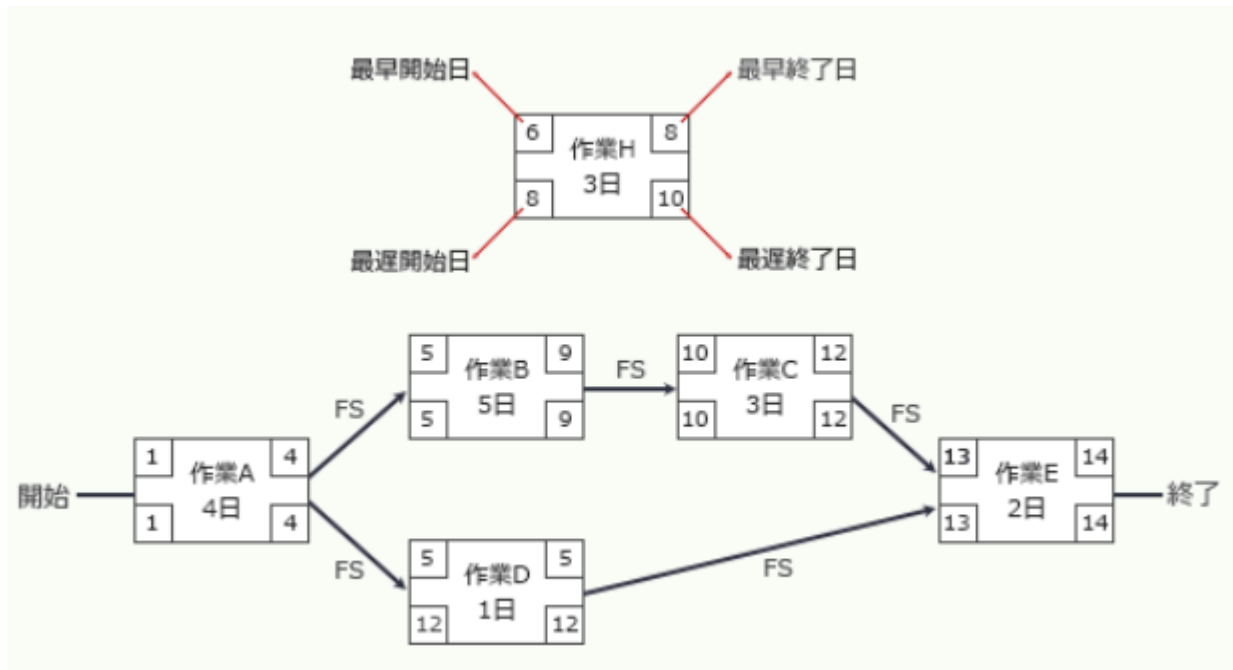


○ 机械式排除选项工：

- 记住，选项中只要出现**刚好是正确答案 2 倍的数字**（如本题的 42 是 21 的两倍），那这个双倍的数字 **100% 是由于忘记除以 2 导致的错误陷阱**，直接打叉排除它，答案必定在剩下的里面！

27. 二つのアクティビティが次の関係にあるとき、論理的な依存関係はどれか。

"システム要件定義プロセス"が完了すれば, "システム方式設計プロセス"が開始できる。



本题考查的是项目进度管理中，两个活动（アクティビティ）之间的**论理的依存関係（逻辑依赖关系）**。这是网络图技术（PDM / 单代号网络图）的核心基石。

考纲规定活动的逻辑关系一共有 4 种。其命名逻辑非常机械化：**前一个活动的性质（F/S） + 后一个活动的性质（F/S）**。

- **F（Finish / 完了）**：代表活动的“结束”。
- **S（Start / 開始）**：代表活动的“开始”。
- **FS関係（Finish-to-Start）**：前一个活动**完了**，后一个活动才能**開始**。
- **FF関係（Finish-to-Finish）**：前一个活动**完了**，后一个活动才能**完了**。
- **SS関係（Start-to-Start）**：前一个活动**開始**，后一个活动才能**開始**。
- **SF関係（Start-to-Finish）**：前一个活动**開始**，后一个活动才能**完了**。

## 🔍 选项解析与详细说明

我们直接切入题干给出的核心日文原句，像医生做手术一样精准拆解它的结构：

「"システム要件定義プロセス"が完了すれば, "システム方式設計プロセス"が開始できる。」

（如果“系统需求定义流程”**完了**，“系统方式设计流程”就可以**開始**。）

- **前置活动**：システム要件定義プロセス → 要求的状态是「完了 (Finish)」。
- **后置活动**：システム方式設計プロセス → 触发的动作为「開始 (Start)」。

因此，这个依赖关系是从前者的 **F** 指向后者的 **S**，即 **Finish-to-Start (FS関係)**。

- **ア：FF関係(Finish-to-Finish)**

- **彻底拆解**：错误。这代表“系统需求定义”不完工，后者的“方式设计”就绝对不能收尾。但这与题干要求的“能不能开始 (開始できる)”毫无关联。

- **イ：FS関係(Finish-to-Start)**

- **彻底拆解**：【正解】完美对应前置活动的「完了 (F)」与后置活动的「開始 (S)」。
- 在实际软件开发中，这也是最合乎常理、使用率超过 90% 的瀑布流关系。

- **ウ：SF関係(Start-to-Finish)**

- **彻底拆解**：错误。这代表前一个活动一拉开序幕 (Start)，后一个活动就必须立刻谢幕 (Finish)。这和题意完全相反。

- **エ：SS関係(Start-to-Start)**

- **彻底拆解**：错误。这代表“需求定义”一动工，后者的“方式设计”就能同步动工（比如敏捷开发或并行动工）。但题干明确要求的必须等前者“完了 (完了すれば)”，所以排除。

## 小学生级记忆法

### “导演与演员的交接棒”理论

把这两个开发流程，想象成**好莱坞剧组在拍电影时的现场调度**：

- **演员 A (系统需求定义)**：正在拍他的杀青戏。
- **演员 B (系统方式设计)**：在后台化好妆，等戏上场。

**FS 关系 (迎宾与送客)：**

导演拿着对讲机高喊：“**等演员 A 演完、杀青 (Finish)，演员 B 才可以上场、开演 (Start)！**”

这就是最老实的“前人送客，后人迎宾”的顺序。A 不走，B 别想来。

### 秒杀口诀

前看“完了”还是“始”，后看“开始”或“收尾”。

“完了”变 F “开始” S，FS 连线直接对！

### 正确答案

イ

## 考试小秘籍

- 考场无脑关键词连连看（降维打击）：
- 在基本情报的上午题中，这类题属于送分题。你根本不需要去理解“要件定義”或“方式設計”是在干嘛，直接在题干里圈出两个动词：
  - 看到「完了」  $\rightarrow$  划线写下 F。
  - 看到「開始」  $\rightarrow$  划线写下 S。
  - 把它们拼在一起：FS。
  - 秒扫选项，イ 赫然写着 FS関係(Finish-to-Start)，2秒钟直接涂卡，绝不浪费一粒脑细胞！

ある会場で資格試験を実施する際のアクティビティである"受付"と"試験"の依存関係のうち、プレシデンスダイアグラム法(PDM)の開始-終了関係はどれか。

本题考查的是进度管理中 プレシデンスダイアグラム法（PDM / 紧前关系绘图法）的 4 种逻辑关系定义。

这也是很多考生最容易绕晕的地方，今天教练带你用最直观的“连线法”瞬间秒杀它！

## 知识点总结

在 PDM 中，两个作业（活动）之间的逻辑关系由「前一个作业的状态」  $\rightarrow$  「后一个作业的状态」来决定。

官方定义了 4 种标准关系（以“受付（A）”和“試験（B）”为例）：

### a. FS関係（Finish-to-Start / 終了-開始）：

- 前一个作业结束（Finish）了，后一个作业才能开始（Start）。

### b. SS関係（Start-to-Start / 開始-開始）：

- 前一个作业开始（Start）了，后一个作业才能开始（Start）。

### c. FF関係（Finish-to-Finish / 終了-終了）：

- 前一个作业结束（Finish）了，后一个作业才能结束（Finish）。

### d. SF関係（Start-to-Finish / 開始-終了）：

- 本题考查的核心。前一个作业开始（Start）了，后一个作业才能结束（Finish）。

## 选项解析与详细说明

题目要求寻找「開始-終了関係（Start-to-Finish / SF関係）」。

我们的唯一判定标准就是：前半句必须是「～の開始」，后半句必须是「～を終了する」。

- **ア：受付の開始から30分経過したら，試験を開始する。**
  - **彻底拆解：**前半句是“受付の**開始**”，后半句是“試験を**開始**”。所以这是 **SS関係（開始-開始）**。
- **イ：受付の終了から10分経過したら，試験を開始する。**
  - **彻底拆解：**前半句是“受付の**終了**”，后半句是“試験を**開始**”。这是最常见的 **FS関係（終了-開始）**。
- **ウ：受付の終了から45分経過したら，試験を終了する。**
  - **彻底拆解：**前半句是“受付の**終了**”，后半句是“試験を**終了**”。这是 **FF関係（終了-終了）**。
- **エ：試験の開始から20分経過したら，受付を終了する。**
  - **彻底拆解：【正解】。**前半句是“試験の**開始**”，后半句是“受付を**終了**”。完美对应 **SF関係（開始-終了）**。
  - **实际场景理解：**考试一旦**开始**（考生全进考场了），门口的“受付（签到迎宾）”工作就可以正式**结束**收摊了。

## 小学生级记忆法

### “交警换班”理论

SF関係（開始-終了）在现实中比较少见，经常让人觉得反直觉。我们用**交警换班的故事**来秒懂它：

有两个交警：**白班交警（受付）** 和 **夜班交警（試験）**。

- **规定：**白班交警绝对不能擅离职守，直到夜班交警来到岗位**开始上班（試験の開始）**，白班交警才可以**下班回家（受付の終了）**。

这就是“你来了（開始），我才能走（終了）”的SF牵绊（选工）！

### 秒杀口诀

PDM考关系，前后动词连成线。FS、SS莫选错，题目要找SF恋。前半开始后半终，一眼看中工选项！

## 小学生级记忆法

### “交警换班”理论

SF関係（開始-終了）在现实中比较少见，经常让人觉得反直觉。我们用**交警换班的故事**来秒懂它：

有两个交警：**白班交警（受付）** 和 **夜班交警（試験）**。

- 规定：白班交警绝对不能擅离职守，直到夜班交警来到岗位开始上班（試験の開始），白班交警才可以下班回家（受付の終了）。

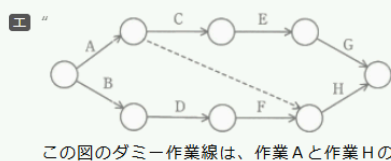
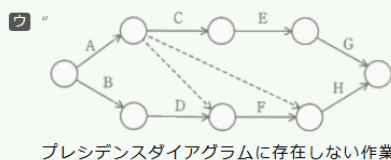
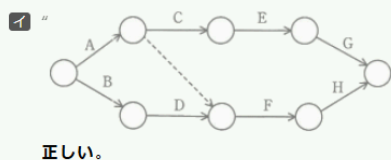
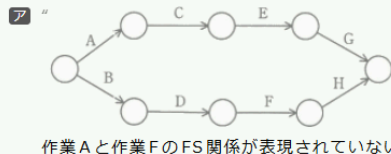
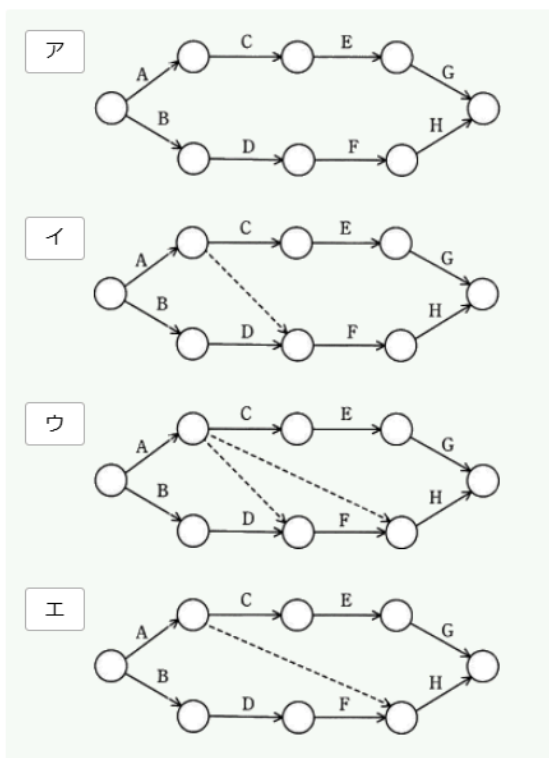
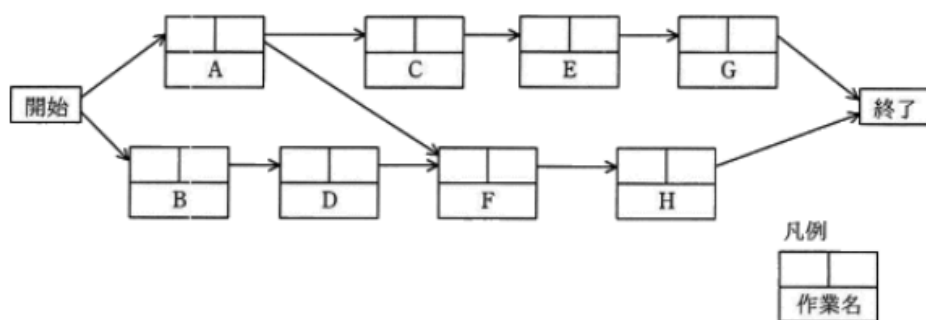
这就是“你来了（開始），我才能走（終了）”的SF牵绊（选工）！

## 秒杀口诀

PDM 考关系，前后动词连成线。FS、SS 莫选错，题目要找 SF 恋。前半开始后半终，一眼看中工选项！

### 28. 见图

次のプレシデンスダイアグラムで表現されたプロジェクトスケジュールネットワーク図を，アローダイアグラムに書き直したものはどれか。ここで，プレシデンスダイアグラムの依存関係は全てFS関係とする。



本题考查的是项目进度管理中，两种核心网络图之间的書き直し（转换与改写）。

#### 1. プレシデンスダイアグラム（PDM / 单代号网络图）：

- 特点：ノード（节点/方框）代表作业（作業），箭头（矢印）仅代表作业之间的逻辑依赖关系。

- **本题已知：**所有的箭头都是 **FS 関係**（前一个干完，后一个才能开始）。

## 2. アローダイアグラム（ADM / 双代号网络图）：

- **特点：**箭头（矢印）代表作业（作業），而圆圈（ノード）仅代表一个时间上的“事件点（Event）”，不消耗任何时间和资源。

## 3. ダミー（虚工作/虚箭头）的本质（考试逻辑核心）：

- 在 ADM（双代号网络图）中，有时为了表达复杂的交叉依赖关系，需要引入「ダミー（虚箭头，虚线表示）」。
- **虚箭头的规则：**它不代表任何实际作业，不花时间。它唯一的任务就是把前一个箭头的“结束点”，连接到后一个箭头的“开始点”，实现逻辑上的依赖传递。

## 选项解析与详细说明

我们用“透视镜”来对比上下两条作业线的依赖关系，揪出隐藏的虚线：

### 第一步：拆解原图的逻辑依赖账本

观察原图（PDM）：

- **上路流水线：** $A \rightarrow C \rightarrow E \rightarrow G$ （一条线串联）。
- **下路流水线：** $B \rightarrow D \rightarrow F \rightarrow H$ （一条线串联）。
- **跨界交叉线：**注意看！从  $A$  的右端 还拉出来了一根箭头，指向了  $F$  的左端。
  - 这意味着： **$F$  这个作业要想开始，不仅要等它前面的  $D$  干完，还必须同时等上路的  $A$  干完！**

### 第二步：在 ADM（箭头代表作业）中还原这个交叉点

在选项的图（ADM）中，作业名（如  $A, B, C, D$ ）都被写在了箭头上。

- 作业  $A$  对应的箭头结束了，会到达一个圆圈（事件点）。
- 作业  $C$  对应的箭头从此圆圈出发。
- 作业  $D$  对应的箭头结束了，到达另一个圆圈。
- 作业  $F$  对应的箭头从此圆圈出发。

因为  $F$  依赖  $A$ ，所以在 ADM 中，必须用一根**虚线箭头（ダミー）**，从“ $A$  箭头的终点（即  $C$  的起点）”指向“ $F$  箭头的起点（即  $D$  的终点）”。

### 第三步：精确狙击选项陷阱

- **ア：**完全没有虚线。这意味着  $F$  仅依赖  $D$ ，丢掉了“ $F$  依赖  $A$ ”的核心逻辑，错误。
- **イ：**虚线从  $A$  的终点指向了  $D$  的起点。这意味着  $D$  依赖  $A$ ，但原图中  $D$  根本不需要等  $A$ ，错误。

- **ウ**：画了两根虚线（从  $A$  到  $H$  的起点，从  $C$  到  $F$  的起点），无中生有，完全乱套，错误。
- **工**：虚线精准地从  $A$  的终点（ $C$  的起点圆圈）跨界指向了  $F$  的起点（ $D$  的终点圆圈）。这完美表达了“ $F$  的动工必须同时取决于  $D$  的完工与  $A$  的完工”。【正解】

## 🧠 小学生级记忆法

### 🚗 “高速公路的跨界高架桥”理论

把这两个网络图，想象成两条并行的单向高速公路：

- **北边高速（上路）**：依次设立了  $A, C, E, G$  四个收费站。
- **南边高速（下路）**：依次设立了  $B, D, F, H$  四个收费站。

交警突然下达了一条特殊限行令（跨界箭头）：

“大货车想要进入南边的  $F$  号收费站，不仅要看它前面的  $D$  站放不放行，还必须等北边  $A$  号收费站的放行通知！”

如何在地图（ADM）上画这个限行命令？

我们必须建一座**秘密高架桥**（ダミー虚线）。

这座高架桥的**入口**，必须设在  $A$  刚结束、准备进  $C$  的那个路口；

这座高架桥的**出口**，必须精准地戳在  $D$  刚结束、准备进  $F$  的那个路口！

顺着高架桥的单向箭头看过去，正好是从  $A$  的尾巴跨到  $F$  的头（选**工**）！

## 💡 秒杀口诀

转换网络看交叉，原图箭头谁指谁。

点名  $A$  来指向  $F$ ，虚线不能乱指挥。

找到  $A$  尾连  $F$  头，无脑锁定选个**工**！

## ✅ 正确答案

**工**

## 📝 考试小秘籍

- **考场3秒“找尾巴”机械判定法**：
    - 这种图示转换题，基准骨架（上下两排字母顺序）在所有选项里长得一模一样。你只需要盯紧原图中那根斜着的跨界箭头。
- 原图里，斜箭头的**起点（尾巴）**是  $A$ ，**终点（箭头）**是  $F$ 。
- 去选项里找虚线：**虚线的尾巴必须在  $A$  后面，虚线的箭头必须指向  $F$  前面。**
- 扫视选项：



ア：没虚线（排除）。

イ：指向了  $D$ （排除）。

ウ：指到了  $H$ （排除）。

エ：精准从  $A$  后指向  $F$  前（秒选）。

- 根本不需要画图，利用“尾巴与箭头”的机械对应，3秒钟直接拿分！

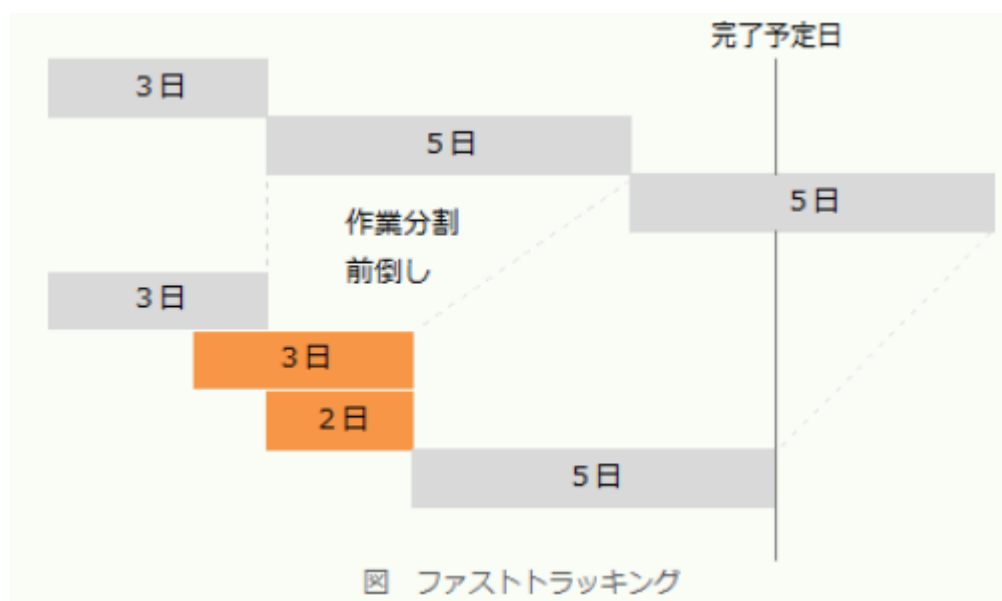
28. 本题考查的是项目进度管理中，当面临工期延误或需要提前完工时，所采用的スケジュール短縮（进度压缩技术）。

考纲规定进度压缩主要有两种核心流派，它们的逻辑完全不同：

a. ファストトラッキング（Fast Tracking/快速跟进）：

- 做法：把原本设计上应该先后顺序执行的作业，改成并行（同时）去干。
- 代价：不增加额外的人手和钱，但因为强行并行，会导致返工（手戻り）风险飙升。

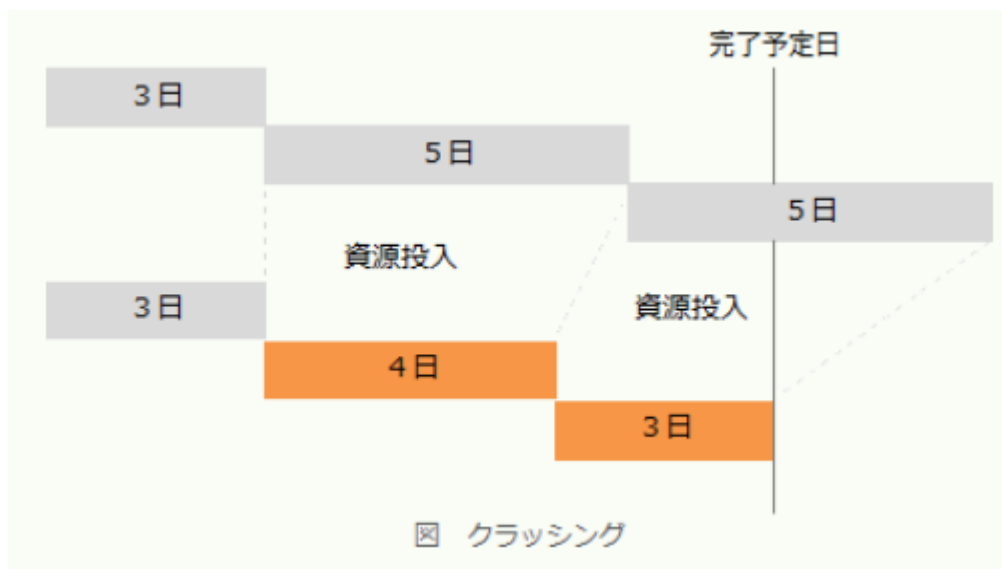
全体の設計が完了する前に仕様が固まっているモジュールの開発を開始するなど、当初の計画では順番に行う予定だったアクティビティを並行して行うことによって短縮を図る方法



b. クラッシング（Crashing/赶工）：

- 做法：也就是本题考查的概念。通过追加投入资源（資源を新たに増員）或让员工加夜班（時間外勤務を増やす）来强行缩短工期。
- 代价：风险小，但项目成本（コスト）会暴涨。

プロジェクトのスケジュールを短縮するために、アクティビティに割り当てる資源を増やして、アクティビティの所要期間を短縮する技法



### c. 关键路径决定总工期（考试逻辑核心）：

- 无论你怎么加人、怎么砸钱，一个项目的总工期完全取决于那条耗时最长的「クリティカルパス（Critical Path/关键路径）」。
- 如果你去给非关键路径上的作业加人，那条最慢的路径依然卡在那里，项目的总完工时间一分钟都不会缩短。因此，投入真金白银赶工时，必须且只能优先投给关键路径。

## 🔍 选项解析与详细说明

我们用“透视镜”来扫描这4个选项，看看谁才是值得砸钱的真命天子：

- **ア：業務の難易度が最も高いスケジュールアクティビティ**
  - 彻底拆解：错误。有些作业虽然很难（難易度が高い），比如设计一个核心加密算法，但它如果在一条时间非常充裕的非关键路径上，你就算加了专家把它提早干完，别的地方还在慢慢吞吞地排队，整个项目依然无法提前完工。
- **イ：クリティカルパス上のスケジュールアクティビティ**
  - 彻底拆解：【正解】关键路径（クリティカルパス）是决定项目什么时候能通关的“瓶颈”。只有缩短了瓶颈上的作业时间，整个大局的完工时间才会真正往前挪。所以砸钱赶工（クラッシング）时，必须雷打不动地优先投给它。
- **ウ：資源が確保できる時期に開始するスケジュールアクティビティ**
  - 彻底拆解：错误。这只是说明该作业什么时候能动工，完全无法保证对缩短整个项目的总工期产生实质性贡献。
- **エ：所要期間を最も長く必要とするスケジュールアクティビティ**
  - 彻底拆解：干扰项。耗时最长（所要期間が最も長く必要）的那个单一作业，不代表它一定在关键路径上（它可能只是一个孤立的、早早动工且不影响大局的长作业）。我们看的是整个路径的累加长度，而不是看谁单独长得胖。

## 🧠 小学生级记忆法

## “奶茶店排队”理论

把整个项目的流水线，想象成你在学校门口开的一家网红奶茶店：

顾客买一杯奶茶需要经过两道并行的流水线：

- **A 线（非关键路径）**：拿个塑料袋和吸管 → 只需要 **10秒**。
- **B 线（关键路径）**：现场煮珍珠、调配奶茶、封口 → 需要 **5分钟**。

现在外面排了100个人，老板说：“我们要搞「クラッシング（赶工）」，多花点工钱多雇两个店员来帮忙，必须把速度提上去！”

- 如果雇来的人去了 **A 线**：店员疯狂拿吸管，1秒钟拿10个。但后面 B 线的奶茶还是要等5分钟。顾客依然拿不到奶茶，老板多付了工钱，却**完全没有缩短顾客排队的时间**。
- 如果雇来的人去了 **B 线**：店员一起帮忙煮珍珠、摇奶茶，把5分钟的瓶颈缩短到了3分钟！顾客拿奶茶的总体速度**立刻变快了**！

这个卡脖子的 B 线，就是「クリティカルパス（关键路径）」。**多花工钱（クラッシング）必须优先用在刀刃上！**

## 秒杀口诀

项目赶工叫 Crashing，大把砸钱请救兵。

救兵不能瞎派兵，只看谁是瓶颈性。

瓶颈要找关键路，クリティカルパス赢（选イ）！

## 考试小秘籍

- 考场纯机械式“拉郎配”关键词锁定法：
  - 在基本情报的所有进度管理题目中，只要题目里出现了「クラッシング（Crashing）」或者「スケジュールの短縮（进度缩短）」。
  - 你的眼睛不需要看别的，立刻去选项里地毯式搜索「クリティカルパス（关键路径）」这个词。
  - 只要找到了，99.9% 盲选它就是正确答案。因为在项目管理理论（PMBOK）里，不针对关键路径的进度压缩方案全都是错误的耍流氓行为！本题 **イ** 完美命中！

29. 本题考查的是 PMBOK 风险管理中，针对已识别风险（Known Risks）所制定的**特定应对策略**。

### ○ コンティンジェンシー計画（应急计划）：

- **本质**：它是一套“触发式”的预案。不是项目一开工就去做的，而是当项目遇到某些特定风险、并且满足了特定的「トリガー（触发条件）」时，才紧急启动的应对措施。
- **例子**：如果室外发布暴雨预警（触发条件），就把发布会立刻移到室内体育馆举行（应急计划）。

## 选项解析与详细说明

我们来拆解选项，看看哪个最符合“应急预案”的特性（由于用户输入的文本存在部分错别字，解析中已自动修正为标准官方题干日文）：

- **ア：あらかじめ定義された、ある条件のときにだけ実行する対応策（预先定义的、只有在满足某种条件时才执行的应对策略）**
  - **彻底拆解：【正解】**。这完美契合了コンティンジェンシー計画的本质定义——“预先备好” + “条件触发”。
- **イ：活動リストの活動ごとに必要な資源（活动列表中每个活动所需的资源）**
  - **彻底拆解：**错误。这属于資源マネジメント（资源管理）或者「アクティビティ資源見積り（估算活动资源）」的范畴，是日常规划的一部分，不是应急计划。
- **ウ：プロジェクトに適用する品質の要求事項及び規格（适用于项目的质量要求及标准）**
  - **彻底拆解：**错误。这属于品質マネジメント（质量管理）中规划质量管理的输出产物，跟风险发生时的应急处理无关。
- **エ：プロジェクトのステークホルダの情報及びコミュニケーションのニーズ（项目利害关系人的信息及沟通需求）**
  - **彻底拆解：**错误。这属于コミュニケーション・マネジメント（沟通管理）中制定沟通管理计划的范畴。

## 小学生级记忆法

### “后备箱里的备胎与千斤顶”理论

把项目管理过程，想象成你和朋友开着一辆越野车去拉萨自驾游：

- **品质与规格（选项ウ）：**出发前规定车速不能超过 120km/h，必须加 95 号汽油。
- **沟通需求（选项エ）：**一路上每过两个小时，必须给家里老妈发个定位报平安。

那么什么是「コンティンジェンシー計画（应急计划）」？就是你在后备箱里锁好的备胎、千斤顶和紧急补胎工具。

- **关键逻辑：**你开车在路上，不能一边开着车，一边每隔 5 分钟就把备胎拿出来抱一抱。你只有在“某一特定条件”发生时——也就是车胎突然被钉子扎爆了（トリガー / 触发条件）——你才会靠边停车，把备胎拿出来用！

这就是“平时锁在后备箱，满足条件才上场”的应急策（选 ア）！

## 秒杀口诀

应急计划计划好，平时不向大家跑。条件触发（ある条件）才执行，ア 项完美对应好！

## 考试小秘籍

◦ 考场纯机械式“条件/触发”关联法：

- 在基本情报的上午题中，只要考到「コンティンジェンシー計画」，你只需要在选项里疯狂肉眼扫描两个日文关键词：

1. 「特定の条件」 / 「ある条件」（特定条件）
2. 「トリガー」（触发器）

- 本题的 **ア** 选项中直接出现了「ある条件のときにだけ実行する」（只在某种条件时执行），属于典型的概念直描。2 秒钟直接锁定 **ア**，安全拿分！

30. PMBOKによれば, プロジェクトのリスクマネジメントににおいて, 脅威に対して適用できる対応戦略と好機に対して適用できる対応戦略がある。脅威に対して適用できる対応戦略はどれか。

| 脅威への対応戦略 | 好機への対応戦略 |
|----------|----------|
| 回避       | 活用       |
| 転嫁       | 共有       |
| 軽減       | 強化       |
| 受容       | 受容       |

本题考查的是 PMBOK（项目管理知识体系）中关于リスク対応戦略（风险应对策略）的分类。

在项目管理中，“风险”是一个中性词，分为两种：

- a. 脅威（きょうい / Threat）：对项目产生负面影响的“坏风险”（消极风险）。
- b. 好機（こうき / Opportunity）：对项目产生正面影响的“好风险”（积极风险/机会）。

PMBOK 针对这两种风险，分别规定了不同的应对策略，它们在考试中有着严格的界限：

◦ 消极风险（威胁）的应对策略：

- 回避（かいひ / Avoid）：彻底消灭风险，概率降为0。
- 軽減（けいげん / Mitigate）：降低风险发生的概率或减轻损失。
- 転嫁（てんか / 移転 / Transfer）：甩锅。把财务损失的责任转让给第三方（如买保险、外包）。
- 受容（じゅよう / Accept）：躺平。不做主动干预，出事再说（保留预备费）。

◦ 积极风险（机会）的应对策略：

- 活用（かつよう / Exploit）：确保机会100%发生。
- 共有（きょうゆう / Share）：把机会分给最能利用它的第三方（如成立合资公司）。
- 強化（きょうか / Enhance）：提高机会发生的概率或增加收益。
- 受容（じゅよう / Accept）：主动或被动地接受机会。

威胁防守四步走：回避、輕減、转嫁（轉嫁）、受容。好机进攻三字经：活用、共有、强化（強化）成。两家受容（受容）是公用，看清威胁（脅威）选防守！

## 考试小秘籍

### ◦ 考场纯字面色彩判定法（降维打击）：

- PMBOK 的术语翻译成日文后，字面的感情色彩极其强烈：
  - 只要题干问「好機に対する戦略」 $\rightarrow$ 去选项里找带有积极、扩大、获取含义的词，如「活用」「強化」「共有」。
  - 只要题干问「脅威に対する戦略」 $\rightarrow$ 去选项里找带有消极、防御、减少含义的词，如「輕減」「回避」「轉嫁」。
- 唯一的特殊情况是「受容」，因为两边都有它（佛系躺平对好坏都适用），所以考试为了避免争议，绝对不会把「受容」拿来当作两者的区分考点。看到「輕減」等防御词，无脑秒选，直接拿下分数！

### 31. テストの進捗管理に使用する指標として、最も適切なものは**テスト項目の消化件数**

ソフトウェアテストは、用意したテストをすべて消化することで、システムやモジュールに含まれる潜在的な欠陥を発見し、ソフトウェアとしての完成度を検証する]工程です。

在软件开发的测试阶段，管理者需要通过量化的数据来把控整个测试团队的节奏。

#### a. 進捗管理（进度管理）的本质：

- 进度管理关心的是“活儿干了多少，还要干多久”。
- **核心指标：テスト項目の消化件数（累计完成的测试用例数/执行件数）**。通过对比“实际消化件数”与“计划消化件数”，就能直观地看出测试进度是超前还是滞后。

#### b. 品質管理（质量管理）的本质：

- 质量管理关心的是“软件做得到底好不好，还有多少隐藏 Bug”。
- **核心指标：バグ発生件数（Bug累计发现数）、不具合密度（缺陷密度）**。通过分析 Bug 的产生曲线来评估软件是否达到了可以出厂的质量标准。

#### テスト項目の消化件数（测试项目消化件数）

- **【进度管理最合适指标】** 它记录的是测试团队每天划掉了多少个待办任务（Test Case）。它的增加只代表“测试工作正在往前推进”，是地道的**进度指标**。

#### バグ発生件数（Bug发生件数）

- **【质量管理指标 / 干扰项】** 不能用来管进度。因为如果某天发现了 0 个 Bug，可能是因为系统质量好（质量高），也可能是因为测试人员那天在摸鱼，一条测试都没跑（进度为 0）。所以 Bug 数只能用来评估**质量健康度**。



测试管理分两家，进度质量别混杂。干活多少看“消化”，进度指标全靠它。Bug 多少看质量，判定准确拿分大！

32. 品質問題を解決するために図を作成して原因の傾向を分したところ、全体の80%以上が少数の原因で占められていることが判明した。作成した図はパレート図

パレート図は、値の大きい順に分析対象の項目を並べた棒グラフと、累積構成比を表す折れ線グラフを組み合わせた複合グラフで、主に複数の分析対象の中から重要である要素を識別するために使用します。

33. システム開発における品質管理に関する記述のうち、適切なものはどれか。

这道题考查的是系统开发中品質管理（质量管理）的核心覆盖范围与基本原则。

### 知识点总结

系统开发中的质量管理并不是只盯着“代码有没有Bug”，它的定义非常广泛：

#### a. 全面质量管理的范围（考试核心）：

- 质量管理的生命周期涵盖整个项目。不仅包括最终交付的软件系统（システム本体），还包括开发过程中产生的所有ドキュメント（设计书、说明书、测试报告等所有成果物）。

#### b. 性能与非功能需求：

- 系统的响应时间、吞吐量等性能（非機能要件）同样是质量的重要指标，绝不是“对象外”。

#### c. 系统集成与第三方组件：

- 将多个子系统或市售产品（市販製品）组合在一起时，必须进行**结合测试和系统测试**。局部质量好不代表整体质量好，所有引入的组件都必须纳入整体质量管理范围内。

### 选项解析与详细说明

- ア：幾つかのサブシステムに分割して開発するとき、サブシステム単位での品質が保証できれば、同時にシステム全体としての品質も保証できる。
  - 彻底拆解：错误。这就是经典的“局部合格不等于整体合格”陷阱。即使每个子系统单独测试（单体テスト）都没问题，但在它们通过接口连接时，依然经常发生数据传递错误或联调失败。因此必须通过システムテスト（系统测试）来保证整体质量。
- イ：応答時間やバッチ処理時間などの性能は品質管理の対象外であるが、業務に与える影響が大きいので限界性能を計測しておく。



- **彻底拆解：**错误。响应时间和批处理时间属于系统的**非機能要件（非功能需求）**，它们是衡量系统好坏的重要质量指标。既然是质量指标，就**属于质量管理的控制对象**，不能说是“对象外”。
- **ウ：システムへの要求機能の充足度だけでなく、ドキュメントなどすべての成果物を含めて品質管理の対象とする。**
  - **【正解】** 质量管理的目标是确保“交付的所有东西都是高品质的”。因此，除了满足用户要求的功能外，开发过程中编写的设计书、用户手册（ドキュメント）等所有**成果物**也必须经过评审（レビュー），全部属于质量管理的对象。
- **エ：市販製品と自社開発プログラムを組み合わせるシステムを開発する場合、品質管理の対象は自社開発のプログラムだけとなる。**
  - **彻底拆解：**错误。虽然市售产品（如商业数据库、中间件）本身的内部Bug我们改不了，但它与自社程序的「組み合わせ（组合/集成）」部分的正确性、兼容性以及整体运行性能，依然必须作为质量管理的对象来进行严格测试。

## 小学生级记忆法

### “造一辆国产豪华轿车”理论

把系统开发过程中的质量管理，想象成**你开了一家汽车制造厂，要打造一辆高品质的轿车：**

- **情况 A（对应选项ア、エ）：** 你从德国引进了最顶级的发动机（市販製品），又自己在车间敲出了一个完美的车壳（自社開発）。
  - **老板说：**“发动机和车壳单拆开都质量爆表，所以组装在一起我连测都不用测，直接卖！”结果一上路，发现车壳卡住了发动机的排气管，车直接炸了。—— **局部好不等于整体好，组合起来同样要管质量！**
- **情况 B（对应选项イ）：** 车子造好了，功能很全，空调音响都能用。但一踩油门，百公里加速要花 1 小时（性能极差）。
  - **产品经理说：**“功能都全了呀，性能差不影响它是一辆好车！”顾客直接退货。—— **性能绝对是质量管理的核心！**

那么什么才叫完美的全面质量管理（选项ウ）？检查员不仅要看车能不能跑（要求機能），连随车赠送的《纸质车主保养手册》（ドキュメント）里有没有错别字，都必须严格检查！任何交到客户手里的成果物，都是质量的一部分（选ウ）！

## 秒杀口诀

质量管理面面到，只看代码立扑街。性能也是硬指标，组合集成跑不掉。文档（ドキュメント）成果全包括，全面管控制胜诀（选ウ）！

## 考试小秘籍

◦ 考场纯机械式“全面、绝对化”语感判定法：

- 在国家考试的质量管理（品質管理）和审计（システム監査）题目中，有一条屡试不爽的心理学规律：
  - 凡是选项中出现「～だけが対象（只有...是对象）」、「～は対象外（...是对象外）」、「～すれば、同時にシステム全体も保証できる（只要...整体就能保证）」这种片面、极端、打包票的描述，99% 都是设计好的错误陷阱项。
  - 凡是选项中表现出「すべてを含めて対象とする（包含所有的成果物作为对象）」、「多角的に評価する（多角度评估）」这种全面、严谨、稳重的描述，基本都是标准正解。
- 本题中，ア（太绝对）、イ（搞排除）、エ（太片面）全部踩雷，唯有 ウ 尽显严谨大厂风范，2秒钟直接秒杀！

34. 本题考查的是项目风险管理中，针对消极风险（威胁）的4大应对策略的字面对应与底层概念。

## 知识点总结

本题考查的是 PMBOK 风险管理中，针对脅威（きょうい / 负面风险）的具体防守策略分类。

当项目中出现可能带来损失的危险时，项目经理有 4 种经典的处置态度：

- a. リスク回避（かいひ / Avoid）：彻底绕着走。改变项目计划，让风险发生的概率直接降为 0。
- b. リスク低減（ていげん / 軽減 / Mitigate）：提前做预防。采取主动措施，降低风险发生的概率或减少可能带来的损失。
- c. リスク移転（いてん / 転嫁 / Transfer）：找人来背锅。把风险带来的财务损失责任转让给第三方（最典型的手段是买保险或外包）。
- d. リスク保有（ほゆう / 受容 / Accept）：也就是本题考查的核心。直接躺平、顺其自然。如果风险带来的潜在损失非常微小，特意花钱去防范反而不划算，于是决定自己承担（受容），出了事直接用项目的预备费来赔钱。

## 选项解析与详细说明

我们紧扣题干的灵魂关键词：「影響が小さい」（影响很小）和「損害の負担を受容する」（接受并自己承担损害的负担）。

◦ ア：リスク移転（风险移转/转嫁）

- 彻底拆解：错误。这代表把损失丢给别人（比如通过买保险或签外包合同），这属于“推卸负担”，而不是题干要求的“自己受容（接受并承担）”。

◦ イ：リスク回避（风险回避）

- 彻底拆解：错误。这代表彻底取消或改变某项原定计划。例如“因为有下雨风险直接取消室外发布会”，这叫回避。题干并没有说要取消计划，所以排除。

◦ **ウ：リスク低減（风险低減/輕減）**

- **彻底拆解：**错误。这需要提前花精力和成本去开发防御机制（比如给系统多做两层加密防止被黑）。题干明确说了影响很小，犯不上花这个冤枉钱。

◦ **エ：リスク保有（风险保有/接受）**

- **彻底拆解：【正解】。**在风险管理理论中，「リスク保有（保有）」与「リスク受容（受容）」在语义上是**完全等价的同义词**。意思就是“这个风险我知道了，影响不大，我自己揣在兜里（保有），出了事我自己认栽（受容）”。

## 小学生级记忆法

### “面对下雨风险的 4 种态度”理论

把项目中可能遇到的风险，想象成你下午准备出门逛街，天空突然飘来了一两片乌云，随时可能会下雨（风险）：

- **风险回避：**你抬头一看，瞬间决定：“算了，不逛街了，我今天宅在家里打游戏！”（彻底取消，概率归零）
- **风险低减：**你虽然还要出门，但未雨绸缪，特意在包里塞了一把结实的大雨伞。（采取预防，减少被淋湿的损失）
- **风险移转：**你没带伞，但你决定花 10 块钱给全身上下的衣服和鞋子买一份“淋湿险”，如果衣服湿了保险公司赔你 200 块。（财务甩锅）

那么什么是「リスク保有 / 受容」（选项工）？你看了一眼天空，发现只是掉了一两点小毛毛雨。你大摇大摆地走入雨中，心里想：“反正就这么两滴雨（影响很小 / 影響が小さい），就算滴到我衣服上，两分钟也就干了，我完全能承受（受容）。我懒得回家拿伞，也懒得花钱买保险，直接肉身抗下这一切（保有）！”

这就是最佛系的风险自理——风险保有（选工）！

### 秒杀口诀

风险应对四战略，字面色彩极强烈。推给保险叫移转，改掉计划叫回避。开发防御叫低减，摊手不管叫保有。题干写明“受容”二，无脑对应“保有”出（选工）！

## 正确答案

工

### 考试小秘籍

◦ **考场纯字面同义词绝对连连看（两秒秒杀法）：**

- 在基本情报以及应用情报的上午题中，“风险保有”这个概念有两个名字，它们在官方考纲（PMBOK）里是 100% 绑定的母子关系：

- 英文原版叫：**Acceptance**。
- 日本国家考试有时候翻译成：「**リスク受容**」。
- 也有时候翻译成：「**リスク保有**」。
- 只要你在题干里看到了「**受容**」这两个汉字，选项里闭着眼睛去找带有「**保有**」的选项；反之，题干里有“保有”，选项里就去找“受容”。
- 这种纯字面映射属于出题官的惯性文字游戏，不需要任何深层思考。看到 **エ** 的 **リスク保有**，2秒钟直接秒杀涂卡！

35. 这是一道考察开发规模估算方法中 **ファンクションポイント法（FP法 / 功能点估算法）** 的经典计算题。只要掌握了它两步走的算账逻辑，考场上就是纯纯的送分题。

## 💡 知识点总结

本题考查的是利用 **ファンクションポイント法（FP法）** 计算最终功能点值的标准流程。

计算一共分为核心的两步：

### a. 未調整ファンクションポイント（未调整功能点，简称 Unadjusted FP）：

- **逻辑**：将各种功能类型的「**個数（个数）**」乘以它们各自的「**重み付け係数（权重系数）**」，然后全部加起来。
- **公式**：
- 未调整 FP =  $\sum(\text{個数} \times \text{重み付け係数})$

### b. 修正（調整）功能点计算（最终ファンクションポイント値）：

- **逻辑**：把算出来的未调整原始分，乘以题目给出的软件整体复杂度的「**補正係数（修正系数）**」。
- **公式**：
- 最终 FP 值 = 未调整 FP × 補正係数

## 🔍 选项解析与详细说明

我们直接根据 **image\_ee5edd.png** 中给出的表格和题干数据，按部就班地进行两步计算：

### 第一步：计算“未调整功能点数”

看图中的表格，把每一行的 **個数 \times 重み付け係数** 算出来并相加：

- **外部入力**：1（個数） × 4（係数） = 4
- **外部出力**：2（個数） × 5（係数） = 10
- **内部論理ファイル**：1（個数） × 10（係数） = 10

未调整功能点总和 = 4 + 10 + 10 = 24

## 第二步：乘以“修正系数”得到最终结果

题干明确指出：「**全体の複雑さの補正係数は0.75とする**」（整体复杂度的修正系数为0.75）。

我们将第一步算出来的24乘以0.75：

最终ファンクションポイント値  $= 24 \times 0.75 = 18$

- **ア：18** —— 【正解】 经过  $24 \times 0.75$  完美计算得出的最终功能点。
- **イ：24** —— 错误。这是第一步算出来的“未调整功能点总和”，考官专门摆在这里测试你有没有忘记乘以修正系数0.75，属于最经典的**半路陷阱项**。
- **ウ：30** —— 错误。干扰数字。
- **エ：32** —— 错误。干扰数字。

## 小学生级记忆法

### “去麦当劳点套餐”理论

把FP法的计算，想象成你和朋友去麦当劳买汉堡和可乐，最后结账时使用打折券的过程：

- **第一步：看菜单算原价（未调整功能点）**

你点了1个麦辣鸡腿堡（外部输入，4元）  $\rightarrow 1 \times 4 = 4$  元。

点了2杯大可乐（外部输出，5元一瓶）  $\rightarrow 2 \times 5 = 10$  元。

点了1个大薯条（内部文件，10元）  $\rightarrow 1 \times 10 = 10$  元。

**原价总计：** $4 + 10 + 10 = 24$ 元。

*（这时候千万别直接掏24元，因为你手里还有一张券！）*

- **第二步：拿出打折券结账（乘以修正系数）**

题干告诉你，今天全场店庆打75折（**修正系数0.75**）。

**最终掏口袋的钱：** $24\text{元（原价）} \times 0.75\text{（打折）} = 18\text{元（选ア）！}$

## 秒杀口诀

功能点法两步走，先乘后加算原额。

一行一行对齐算，加出二十四（24）别急涂。

题目还有折价券，乘上零点七五（0.75）出。

最终十八（18）才是真，稳稳拿下选个ア！

## 考试小秘籍

- **考场心理学破局——警惕“半路数字”：**

- 基本情报的计算题中，出题官非常喜欢把“计算到一半的中间产物”塞进选项里。本题里算出来的 \$24\$（イ）就是一个巨大的诱饵。
- **教练铁律**：只要在题干里看到了「補正係数」或者「トレンド」这种百分比、小数数据，你在草稿纸上算出来的第一个大整数（比如 24），绝对不能直接去选！必须和这个小数乘一下，得到的最终数字（18）才是正确答案。牢记这个心眼，陷阱直接沦为送分福利！

36. 本题考查的是项目风险管理中，用于识别风险（リスク識別）的经典专家调查技术——**デルファイ法（Delphi method / 德尔菲法）**。

德尔菲法的核心思想是“通过匿名的多轮反馈，让多位专家的意见最终收敛、达成一致”。它有两个最关键的灵魂特征：

- 匿名（匿名）**：专家们彼此不知道对方是谁，全靠问卷沟通。这样能防止“权威压死人”或“随大流”。
- 繰り返して収束（反复循环直至收敛）**：组织者把第一轮问卷结果收上来、做个摘要（要約），再发给所有专家（再配布）。大家看完别人的意见后，修改自己的想法。这个过程重复好几次，直到大家的意见基本一致（収束）。

## 选项解析与详细说明

为什么你选的 **ア** 不对？我们来给 4 个选项做个精准的“身份鉴定”：

- **ア：確率分布を使用したシミュレーションを行う。**
  - **为什么不对**：这是 **モンテカルロ法（Monte Carlo method / 蒙特卡洛模拟法）** 的定义。
  - **深度拆解**：题目问的是“风险识别（リスク識別）”阶段的技法。而蒙特卡洛法是利用计算机进行成千上万次概率模拟，用来定量评估项目工期或成本的超支概率，属于“风险定量分析（定量的リスク分析）”阶段的技法。阶段和定义都彻底搞错了。
- **イ：過去の情報や知識を基にして、あらかじめ想定されるリスクをチェックリストにまとめておき、チェックリストと照らし合わせることでリスクを識別する。**
  - **彻底拆解**：错误。这是 **チェックリスト分析（Checklist Analysis / 检查表分析法）**。
- **ウ：何人かが集まって、他人のアイディアを批判することなく、自由に多くのアイディアを出し合う。**
  - **彻底拆解**：错误。这叫 **ブレインストーミング（Brainstorming / 头脑风暴法）**。它的核心是“不批判、自由发言”，但它是大家聚在一起（面对面）大喇叭喊，没有匿名的机制。
- **エ：複数の専門家から得られた匿名の见解を要約して、再配布することを何度か繰り返して収束される。**
  - **【正解】完美命中**。看到了「匿名（匿名）」、「要約して、再配布」以及「繰り返して収束（反复循环直至收敛）」，这全都是德尔菲法写在教科书上的标准定义。

## 风险管理相似技法排查表



为了防止下次再把这几个概念搞混，我们拉出一道防火墙：

| 技法名称 (日语原件)   | 核心沟通形式         | 核心特征词（肉眼搜寻目标）         | 所属管 |
|---------------|----------------|-----------------------|-----|
| デルファイ法        | 专家们背靠背，互不知道是谁  | 匿名、要約・再配布、繰り返して収束     | リスク |
| ブレインストーミング    | 大家面对面，坐在一起头脑风暴 | 集まって、批判することなく、自由に出し合う | リスク |
| モンテカルロ法 (选项ア) | 丢给计算机，用数学公式跑模拟 | 確率分布、シミュレーション         | 定量的 |

## 小学生级记忆法

### “蒙面巫师猜体重”理论

把德尔菲法（デルファイ法），想象成系统选拔赛中，项目经理要预测一个新模块的开发风险有多大。他找来了 3 位行业顶级专家：

如果用头脑风暴（选项ウ）：三个人坐在一起。大佬一开口，另外两个小弟迫于威严只能点头附和：“对对对，大佬说得都对！”——结果流于表面。

于是项目经理启动了「德尔菲法」：

1. 他把 3 位专家关在不同的黑屋子里，让他们互相不知道对方是谁（匿名）。

2. 第一轮：让他们各自写下风险预测值。收上来一看，有人说 10 天，有人说 50 天。

3. 第二轮：项目经理把这三个数据做成一个匿名表格（要約），重新发给三位专家（再配布）。专家 A 看完后心想：“哦？居然有人觉得要 50 天，看来我漏掉了某些复杂的技术坎，那我改成 30 天吧。”

4. 如此反复折腾了几轮（繰り返して），大家的预测终于都稳定在了 35 天左右（収束）。

这种“蒙面背靠背，连发好几轮”的专家扯皮法，就叫德尔菲法（选 工）！

### 秒杀口诀

概率分布模拟跑，那是电脑蒙特卡洛（ア项）。  
不准批判头脑风，那是聚会风暴冲（ウ项）。  
德尔菲法要高冷，蒙面匿名（匿名）发几轮。  
总结分发求收敛（収束），精确定位选个 工！

### 正确答案

工

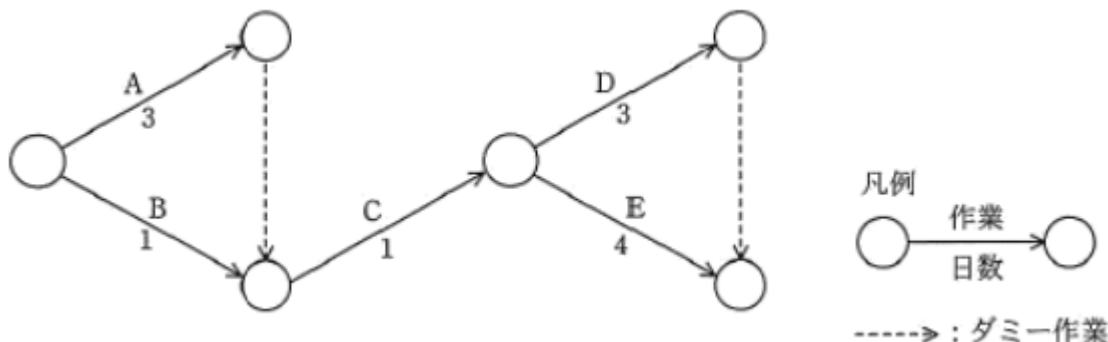
### 考试小秘籍

- 考场纯关键词“连连看”绝对法则：



- 在基本情报下午或上午的风险管理题目中，「デルファイ法（德尔菲法）」的出题套路极其单一。
- 只要你在题干或者选项中，看到「匿名」和「繰り返（す）」/「収束」这两组词同时出现，不需要有任何犹豫，它们之间是 100% 的互锁关系。
- 至于「確率分布」和「シミュレーション」（选项ア），它们只和「モンテカルロ法」绑定。看清这个套路，下次考场直接拿捏！

37. 本题表面考查的是 **アローダイアグラム（箭线图/双代号网络图）**，但核心考点是 **リソースの平準化（资源平衡）与工期最优化演练**。



a. 图表的基本依赖账（看懂箭头）：

- 虚线是 **ダミー作業（虚工作）**，不花时间。
- **前半段逻辑**：\$C\$ 要想开始，必须等 \$A\$（3天）和 \$B\$（1天）**全部干完**。所以前一阶段必须等耗时最长的 \$A\$ 完工。
- **后半段逻辑**：整个项目要完工，必须等 \$D\$（3天）和 \$E\$（4天）**全部干完**。

b. 特殊约束规则（考试的核心瓶颈）：

- **原本计划**：只有 **1个人（主力）** 包揽所有活。
- **突发福利**：第 2 天到第 6 天（共 **5 天时间**），多了一个**帮手（助手）**。
- **死命令**：**1个作业不能 2 个人同时干**（不能男女搭配干活不累）。但一个人干到一半，另一个人可以**过来接班（引き継ぐ）**。

## 🔍 选项解析与详细说明

我们采取“两步对比法”，算出原本的工期，再给两个人安排最完美的“极限排班”：

### 第一步：计算【没有帮手】时，一个人干需要多少天？

如果只有主力 1 个人，因为分身乏术，所有的作业只能一个接一个、串行去干。

- **前半段**：\$A\$ 耗时 3 天，\$B\$ 耗时 1 天。虽然它们在图上是并行的，但 1 个人必须先花 3 天做 \$A\$，再花 1 天做 \$B\$。所以做完 \$A\$ 和 \$B\$ 一共要花  $3 + 1 = 4$  天。此时 \$C\$ 才可以动工。
- **中段**：\$C\$ 耗时 1 天。

- 后半段：  $D$  耗时 3 天，  $E$  耗时 4 天。1 个人必须先花 3 天做  $D$ ，再花 4 天做  $E$ 。一共花  $3 + 4 = 7$  天。

 没有帮手时的总天数 =  $(3 + 1) + 1 + (3 + 4) = 12$ 天。

## 第二步：计算【有帮手在第 2~6 天加入】时的极限最短天数

现在，帮手在第 2 天、第 3 天、第 4 天、第 5 天、第 6 天（共 5 天）在场。我们要让这两台“发动机”一分钟都不闲着：

- 第 1 天：只有【主力】1人。
  - 主力开始做  $SA$ （完成  $SA$  的第 1 天）。
- 第 2 天：（★帮手加入！）现在有 2 个人。
  - 主力继续做  $SA$ （完成  $SA$  的第 2 天）。
  - 帮手去开辟新战场，做  $SB$ （ $SB$  只需要 1 天， **$SB$  彻底完工！**）。
- 第 3 天：有 2 个人。
  - 主力继续做  $SA$ （完成  $SA$  的第 3 天， **$SA$  彻底完工！**）。
  - 此时，由于  $SA$  和  $SB$  都干完了， **$SC$  具备了动工条件**。帮手立刻无缝衔接去干  $SC$ （ $SC$  需要 1 天， **$SC$  彻底完工！**）。
  - 这一天结束，前半段和中段全部清空！
- 第 4 天：有 2 个人。进入后半段（有  $D$  和  $E$  两个任务）。
  - 主力开始做  $SE$ （完成  $SE$  的第 1 天）。
  - 帮手开始做  $SD$ （完成  $SD$  的第 1 天）。
- 第 5 天：有 2 个人。
  - 主力继续做  $SE$ （完成  $SE$  的第 2 天）。
  - 帮手继续做  $SD$ （完成  $SD$  的第 2 天）。
- 第 6 天：（★帮手最后一天在场！）有 2 个人。
  - 主力继续做  $SE$ （完成  $SE$  的第 3 天）。
  - 帮手继续做  $SD$ （完成  $SD$  的第 3 天， **$SD$  彻底完工！**）。
  - 下班后，帮手吃饱喝足撤离剧组。
- 第 7 天：又只剩【主力】1人。
  - 主力独自收尾，做  $SE$  的最后一天（完成  $SE$  的第 4 天， **$SE$  彻底完工！**）。

 有帮手时的最短总天数 = 7天。

## 第三步：计算「最大で何日短くなるか」（工期缩短了几天？）

千万不要一激动直接去选 7！题目问的是**比原来缩短了几天**：

缩短的天数 = 原工期 (12天) - 新工期 (7天) = 5天

## 考试小秘籍

### ◦ 考场纯机械式“资源效率100%利用”蒙题规律：

- 这类资源平衡题，出题官设计“帮手来了  $\$X\$$  天”时，通常会把这个排班流程卡得极为精妙。
- 只要我们在脑海里推演一遍，发现**帮手在场的那 5 天里（第2天到第6天），天天都有别的活可以让她干，她一分钟都没有闲着（手空きがない）**。
- 既然帮手 100% 全程在满负荷输出，并且没有发生任何人力的浪费（没有出现 2 个人抢同一个作业），那么**她输出的 5 天纯劳动力，就会完完全全转化为缩短的项目总工期（也就是 5 天）**！
- 掌握这个终极效率内核，在考场上甚至可以跳过繁琐的按天推导，直接对应帮手在场的 5 天时间，秒选 **ウ**！

38. 本题考查的是软件工程中至关重要的 **ソフトウェア品質特性（软件质量特性）** 国际标准（旧 ISO/IEC 9126，即 JIS X 0129 标准）。

标准中规定了软件质量的 **6大主特性**。考试时，出题官通常会给出一个非常具体的开发场景或描述，让你判断它属于哪一个特性：

- a. **機能性（Functionality）**：软件能不能满足用户的明确需求、功能齐不齐全。
- b. **信頼性（Reliability）**：软件稳不稳定，遇到错误时能不能不崩溃、继续正常运行。
- c. **使用性（Usability）**：软件好不好用，界面容不容易理解，用户上不上手。
- d. **効率性（Efficiency）**：软件运行得快不快，吃不吃内存和 CPU 资源。
- e. **保守性（Maintainability）**：软件出了 Bug 好不好改，后续方不方便调整或升级。
- f. **移植性（Portability）**：【**本题核心**】软件能不能轻松地从一个运行环境（硬件、操作系统、架构）挪到另一个不同的环境下继续跑。

## 选项解析与详细说明

我们紧扣题干的灵魂关键词：「**別のアーキテクチャのコンピュータで動作できる**」（在其他不同架构的计算机上也能正常动作）以及「**容易さ**」（简易程度）。

### ◦ **ア：移植性（Portability）**

- **彻底拆解：【正解】**。这里的“移”就是移动、“植”就是栽种。把在一台电脑上写好的软件，“移植”到另一种完全不同架构或操作系统的电脑上运行，这种容易程度在标准定义中就是 **移植性（Portability）**。

### ◦ **イ：使用性（Usability）**

- **彻底拆解**：错误。它衡量的是用户在操作这个软件时的好用程度。比如“按钮好不好点”、“菜单容不容易懂”，与更换计算机硬件平台无关。

### ◦ ウ：相互運用性（Interoperability）

- **彻底拆解**：错误。它指的是两个或多个不同的软件/系统之间，能不能互相传输数据、顺利地协同工作（通常翻译为互操作性）。题干强调的是单个程序换硬件换环境跑，而不是系统间手拉手通信。

### ◦ エ：变更性（Changeability）

- **彻底拆解**：错误。它是“保守性（Maintainability）”下面的一个子特性，指的是代码出了错或有了新需求时，**修改代码本身的容易程度**。题干说的是“去别的架构上跑”，没说要大幅修改代码。

## 软件质量特性考场秒杀对应表

为了让你在考场上不仅能做对这道题，还能顺便干掉其他5个特性的变形题，请牢记以下高频关键词映射：

| 主特性 (日文/英文)           | 子特性举例 (日文) | 考场灵魂核心词 (肉眼搜寻目标)    | 典型大白话场景                 |
|-----------------------|------------|---------------------|-------------------------|
| 移植性 (Portability)     | 環境適応性、設置性  | 別のアーキテクチャ、別のOS、環境移行 | 把 Windows 软件搬到 Mac 上跑   |
| 使用性 (Usability)       | 理解性、習得性    | ユーザーが理解しやすい、操作の容易さ  | 界面设计得极其傻瓜化，一眼看懂         |
| 保守性 (Maintainability) | 变更性、解析性    | バグ修正、プログラムの変更のしやすさ  | 代码写了详细注释，以后改起来简单        |
| 機能性 (Functionality)   | 合目的性、相互運用性 | 他のシステムとのデータ交換、機能満足  | 能够完美读取和导入别家系统的 Excel 报表 |

## 考试小秘籍

### ◦ 考场纯字面对应法则（秒杀技巧）：

- 在信息安全或系统开发上午题中，只要看到「別の～で動作」、「他のプラットフォーム」（其他平台）、或者「異なる環境に移行」（移到不同环境），在选项中不需要多想，百分之百直接锁死「移植性」。
- 另外注意，「相互運用性」（乌项）的考点固定对应「他のシステムと～」（与其他系统协同）。分清这俩“移”字辈和“用”字辈的差别，这一分稳稳装进兜里！

39. 本题考查的是项目管理中**沟通渠道数量（コミュニケーションチャンネル数）**的经典计算，在数学上属于典型的**组合问题（組み合わせ）**。

题目：プロジェクトメンバーが16人のとき、2人ずつの総当たりでプロジェクトメンバー相互の顔合わせ会を行うためには、延べ何時間の顔合わせ会が必要か。ここで、顔合わせ会1回の所要時間

は0.5時間とする。

### 核心法则：

当有  $n$  个人需要\*\*两两不重复地组队见面（総当たり）\*\*时，总共需要的见面次数（配对数）公式为：

$$\text{总见面次数} = \frac{n \times (n - 1)}{2}$$

算出总次数后，再乘以**单次见面所需的时间（所要時間）**，就能得到最终的总小时数。

## 选项解析与详细说明

我们直接带入题目给出的数字，分两步列出明白账：

### 第一步：计算一共有多少对人需要见面？

- 题目给出项目成员总数  $n = 16$  人。
- 代入公式计算总见面次数：
- 总次数  $= \frac{16 \times (16 - 1)}{2} = \frac{16 \times 15}{2} = 8 \times 15 = 120$ 次

### 第二步：计算累计（延べ）需要多少小时？

- 题目明确说明：每 2 个人见一次面，需要花 **0.5 小时**（也就是 30 分钟）。
- 那么 120 次见面总共消耗的时间为：
- 总时间  $= 120\text{次} \times 0.5\text{小时} = 60\text{小时}$

## 小学生级记忆法

### “世界杯小组赛踢循环赛”理论

把这 16 个项目成员，想象成 **16 支足球队**要在小组赛里打单循环赛（每两支队都要踢一场，即総当たり）：

#### 算场次：

第一支队要和剩下的 15 支队各踢一场（15场）。因为每场比赛是两队一起踢，为了不重复计算，16 支队总共要踢的场次就是  $\frac{16 \times 15}{2} = 120$  场比赛。

#### 算时间：

组委会规定，为了节省球员体力，**每场比赛只踢半小时（0.5 小时）**。

#### 总工作量：

120 场比赛，每场半小时。如果只有一块场地，市长要来看完所有的比赛，一共要在主席台上坐多久？

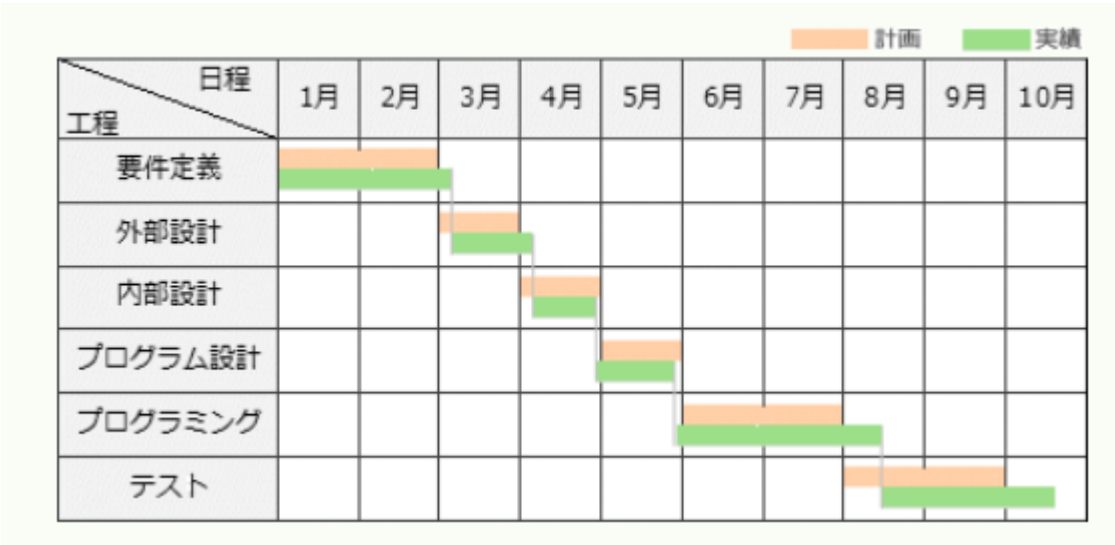
120 × 0.5 = 60小时

 考试小秘籍

- 两步走的“机械化解题法”：
- 信息处理安全技术者考试中，只要看到「メンバーがX人」、「総当たり」（或者「コミュニケーションチャンネル」）这两个关键词，在草稿纸上不要犹豫，直接做两件事：
  - i. 用  $\frac{\text{人数} \times (\text{人数} - 1)}{2}$  算出总配对数。
  - ii. 看清题目最后求的是“次数”还是“总时间”。如果求总时间，务必乘以单次时间（比如这道题的 0.5），即可无脑锁定正确选项！

40. 進捗管理に用いられるガントチャートの特徴として,日程について予定と実績を対比することができる。

ガントチャート(Gantt chart)は、縦軸に作業項目・横軸に時間を取り、作業に必要な期間を横棒の長さで表記する図法です。作業項目同士の日程的な重なりがわかりやすいため日程計画の策定に使用されたり、計画と実績を並べて表記することで進捗度を管理する方法としても用いられます。



41. 本题考核的核心考纲是「工数計算（こうすうけいさん）」，即项目工作量估算。
- **核心单位：人日（にんにち）**。1人日代表“1个人干1天的工作量”。如果是10人日，可以由1个人干10天，也可以由10个人干1天。
  - **整体思维：系统的全体工数（总工作量）** = コーディング（编码） + 設計（设计） + テスト（测试）。在估算总成本和总人数时，必须把所有的环节都加起来，缺一不可。

 选项解析与详细说明

这道题的“杀手锏”在于这句话：「コーディングのほかに、設計及びテストの作業が必要であり、それらの作業にはコーディング所要工数の8倍の工数が掛かる」（除了编码之外，还需要设



计和测试，这些工作需要编码工作量**8倍**的工时。)

我们来一步步拆解：

a. 先算出基础的「コーディング（编码）」总工时：

- 入力処理：20本 × 1人日 = 20人日
- 出力処理：10本 × 3人日 = 30人日
- 計算処理：5本 × 9人日 = 45人日
- 编码总工时 = 20 + 30 + 45 = **95人日**

b. 打破陷阱（辨析选项）：

- 【干扰项】ア 8： 很多考生算到编码是 95人日，看到题目说“8倍”，直接  $95 \times 8 = 760$ 人日。然后用  $760 \text{人日} \div 95 \text{天} = 8 \text{人}$ 。**这是死路！** 因为这 760人日仅仅是“设计和测试”的工作量，你把写代码（编码）的工作量给扔了！光设计不写代码，系统能跑吗？
- 【正确项】イ 9： 正确的逻辑是，总工作量 = 编码（1份） + 设计与测试（8份） = **9份**。所以总工时是  $95 \text{人日} \times (1 + 8) = \mathbf{855 \text{人日}}$ 。要在 95天（日間）内完成，需要的「要員（人员）」 =  $855 \text{人日} \div 95 \text{天} = \mathbf{9 \text{人}}$ 。
- ウ 12 & エ 13： 纯粹的凑数干扰项，用来迷惑那些计算过程中加减乘除算错的考生。

システムを構成するプログラムの本数とプログラム1本当たりのコーディング所要工数  
おりであるとき、システムを95日間で開発するには少なくとも何人の要員が必要か。  
システムの開発にはコーディングのほかに、設計及びテストの作業が必要であり、そ  
業にはコーディング所要工数の8倍の工数が掛かるものとする。

|      | プログラムの<br>本数 | プログラム1本当たりの<br>コーディング所要工数（人日） |
|------|--------------|-------------------------------|
| 入力処理 | 20           | 1                             |
| 出力処理 | 10           | 3                             |
| 計算処理 | 5            | 9                             |

 解题计算步骤表

在考场上遇到这类题，请立刻在草稿纸上画出这样的三步表格，绝不会错：

| 步骤  | 目标            | 计算过程                                                  | 结果           |
|-----|---------------|-------------------------------------------------------|--------------|
| 第一步 | 算基准（コーディング工数） | $(20 \times 1) + (10 \times 3) + (5 \times 9)$        | <b>95人日</b>  |
| 第二步 | 算总计（システム全体工数） | 编码(1倍) + 设计测试(8倍) = 总计(9倍)<br>$95 \text{人日} \times 9$ | <b>855人日</b> |
| 第三步 | 算人数（必要な要員）    | 总工时(855) ÷ 开发天数(95)                                   | <b>9人</b>    |



(小提示：其实第二步和第三步可以合并，总工时 =  $95 \times 9$ ，除以天数 95，95和95直接抵消，心算秒出答案 9)

## 🧠 小学生级记忆法

让我们把枯燥的软件工程变成“盖房子（建房工程）”：

- **コーディング（写代码）**：就像是“搬砖砌墙”。
- **設計及びテスト（设计和测试）**：就像是“画图纸和搞精装修”。

题目说：“除了搬砖砌墙，我们还要画图纸和精装修，装修的工作量是搬砖的**8倍**哦！”

如果老板问你一共需要多少人力的工资？

你绝对不能只算装修的钱（8倍），还要把搬砖师傅的钱（1倍）算进去啊！

所以，**总工程量 = 搬砖（1） + 装修（8） = 9倍！**

以后看到这种题，直接在题目上的“8倍”旁边写上一个大大的“+1”！

## 📝 考试小秘籍

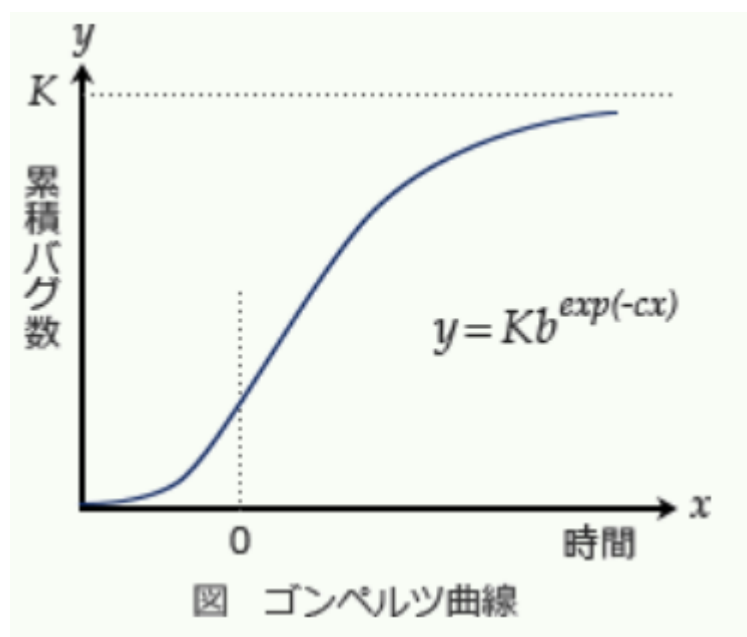
**考场机械式判断技巧：**

只要在日语原题中看到「～のほかに（除了...之外）」加上「**〇倍**」这个组合句式，大脑请立刻启动【加1法则】！

不要犹豫，总工作量的乘数永远是 **（〇 + 1）**。

比如题目说“是测试的 3 倍”，那总工作量就是乘 **4**；说是“编码的 8 倍”，总乘数就是 **9**。直接用基础数字乘上这个“+1后的倍数”，精准避开出题人的坑！

42. ペルツ曲線とは、以下のようなS字曲線です。



本题考核的核心概念是「信頼度成長曲線（しんらいどせいちょうきょくせん）」，也常被称为「ゴンペルツ曲線（Gompertz curve / S字曲线）」。

- **X轴（テスト項目消化件数）**：你执行了多少个测试用例（进度）。
- **Y轴（累積バグ件数）**：你**累计**发现了多少个Bug。请注意“累计”二字，它意味着数值永远不可能倒退（变小）。
- **核心逻辑**：软件质量变好（品質が安定），并不是指“没有Bug”，而是指“不管怎么继续测，都很难再找出新的Bug了”。在图表上，这就表现为代表累计Bug数的线条趋于平缓（不再上升）。

### 选项解析与详细说明

我们来逐一打破选项的陷阱：

- **【干扰项】ア**：曲线越往后越陡峭。这意味着你越测，发现新Bug的速度越快！这说明软件简直是个“马蜂窝”，到处是坑，**品质极其恶劣**，完全没有稳定的迹象。
- **【干扰项】イ**：曲线在下降。这是一个**违背常识的逻辑陷阱**！Y轴是「累積（累计）」，累计数量只会增加或不变，绝对不可能减少（除非你时光倒流让Bug消失）。这个图本身就是错的。
- **【干扰项】ウ**：呈一条直线持续上升。这表示你每做一个测试，就会稳定地发现固定数量的Bug。说明修Bug和找Bug陷入了死循环，系统依然不稳定。
- **【正确项】エ**：经典的S字曲线。初期找Bug较慢（可能在搭建环境）；中期直线飙升（大量测试疯狂发现Bug）；**末期线条变平**。线条变平意味着：虽然「テスト項目（测试用例）」还在继续往下做（X轴在走），但「累積バグ件数（累计Bug数）」不再增加了（Y轴不动了）。这完美对应了题目要求的「品質が安定しつつある（质量正在趋于稳定）」。

### 软件测试阶段排查表

为了让你在考场上彻底理解S曲线的逻辑，我们可以把它拆解为三个必经阶段：

| 测试阶段 | 图表表现 (工选项) | 实际工作场景                            | 质量   |
|------|------------|-----------------------------------|------|
| 初期   | 缓慢上升       | 刚开始测试，可能环境没配好，或者测试人员还在熟悉系统。       | 尚未   |
| 中期   | 陡峭上升       | 测试全面铺开，大家疯狂揪出各种潜藏的Bug，累计数量飙升。     | 极度   |
| 末期   | 趋于平缓（横向打平） | 能找的Bug基本都找完了。测试还在进行，但很久才冒出一个新Bug。 | 趋于安定 |

### 小学生级记忆法

让我们把枯燥的软件测试变成“在密闭房间里打蚊子”：

- **テスト項目消化件数（X轴）**：你花在房间里找蚊子的时间。
- **累積バグ件数（Y轴）**：你**总共打死**的蚊子数量。

一开始（初期），你刚进房间，还没摸清蚊子在哪，打得少。

接着（中期），你找到了窍门，疯狂拍打，“啪啪啪”，总击杀数直线飙升！

最后（末期），房间里的蚊子几乎被你消灭光了。你依然在房间里到处找（X轴时间增加），但半天也打不到一只新的了，你的**总击杀数**再怎么增加了（**线条平缓**）。

这时候，你就可以安心去睡觉了，因为这个房间的“品质（无蚊环境）已经稳定了”！

## 考试小秘籍

### 考场机械式判断技巧：

在考试中，只要看到题目里同时出现「テスト（测试）」、「バグ（Bug）」和「品質が安定（质量稳定）」这三个关键词，不要有任何犹豫，立刻去选项里找那条“字母S形状”且“尾部平躺”的曲线（ゴンペルツ曲線）！尾巴平躺，意味着考试稳过！

43. 開発工数は、開発規模÷生産性で計算されるので、見積りには単位時間当たりの生産する生産性が必要な情報となります。

ソフトウェアの開発規模から開発工数を見積もる際に、必要な情報は生産性

44. 人月（にんげつ）是软件工程和项目管理中衡量「工数（こうすう - 工作量/工程量）」的最核心单位。
- 它既不代表“人数”，也不代表“月数”，它代表的是一个「固定的工作总量」。
  - 定义：1人月 = 1个人全职工作1个月所能完成的工作量。
  - 核心公式：工数（人月） = 要員（人数） × 期間（月数）
  - 在考试中，看到“人月”，第一反应永远不要把它拆开看，它就是一个「总能量槽」或者「工作总数」。

## 选项解析与详细说明（思维误区拆解）

为了让你彻底不再迷茫，我们把考生遇到“人月”时最容易掉入的思维陷阱当作“干扰项”，进行彻底拆解：

- 【误区陷阱一】看到“12人月”，下意识认为这个项目必须干12个月，或者必须找12个人。
  - 教练解析：这是最大的误区！12人月是一个可以任意揉捏的“橡皮泥”。  
如果只有1个人干，需要干12个月（ $1人 \times 12个月 = 12人月$ ）  
如果有2个人干，需要干6个月（ $2人 \times 6个月 = 12人月$ ）  
如果有4个人干，只需要干3个月（ $4人 \times 3个月 = 12人月$ ）
- 【误区陷阱二】认为人月可以无限倒推，比如“12人月”的项目，找360个人干1天就能完工。
  - 教练解析：在数学计算上看似成立，但在实际考试中（常考的下午题或管理理论题），会考核「人月的局限性」。人多了需要沟通成本，有些工作无法并行（比如1个孕妇怀胎10个月生娃，你找10个孕妇不可能1个月把娃生出来）。
- 【正确解题思维】看到人月，立刻把它看作一个由“人数”和“月数”相乘得到的“面积”。

- **教练解析：**只要题目给你了“面积（人月）”，再给你一条边（比如限制了几个月完工，或者限制了只有几个人），你直接用除法就能求出另一条边。



## 看到“人月”的考场第一步排查表

每次在题目里看到「人月」或者需要算「工数」时，请立刻在草稿纸上执行以下三步排查：

| 动作       | 核心思考                                                       | 目的                         |
|----------|------------------------------------------------------------|----------------------------|
| 锁定“蛋糕总量” | 题目有没有直接给出总工数是多少「人月」？<br>如果没有，先用 <b>人数</b> × <b>月数</b> 算出来。 | 把虚无的概念变成一个固定的、不变的“总工作量数字”。 |
| 看清“题目限制” | 题目要求的最终问题是问「何人必要か（需要多少人）」还是「何ヶ月かかるか（需要多少个月）」？              | 确定你要把这个总量拿去除以谁。            |
| 执行除法大刀   | • 问人数：人数 = 总人月 ÷ 限定月数<br><br>• 问时间：月数 = 总人月 ÷ 现有人数         | 直接切中要害，秒杀选项。               |



## 小学生级记忆法

我们把“人月”想象成「一块巨大的生日蛋糕」：

- 项目经理说：“这个系统的开发工数是 **12人月**。”
- 意思就是：“厨房里做了一块需要**12个人**合力吃**1个月**才能吃完的特大蛋糕。”
- **如果只有你一个人吃（1人独立开发）：**你肚子容量有限，只能老老实实连着吃 **12个月** 才能把蛋糕消灭。
- **如果你叫了3个好朋友一起吃（4人团队开发）：**大家平摊，每个人吃自己的那份，**3个月** 就把这块大蛋糕吃光了。
- **如果老板在后面拿鞭子抽你，要求必须在2个月内吃完（限制工期2个月）：**为了不撑死，你必须立刻打电话叫人，让桌子周围坐满 **6个人** 一起吃（ $12 \div 2 = 6$  人）。

所以，看到人月，它就是那块**固定大小的蛋糕**！人数变多，吃的时间就变短；时间缩短，要来的人数就得变多！



## 正确答案（核心心法）

每次看到人月，你先要做的事情就是：「在心里把它转变成一个纯粹的乘法数字，然后寻找题目给你的除数（人数或月数）」。

### 考试小秘籍

#### 考场机械式判断技巧：

- a. 看到「人月」，在题目旁边写下大大的： $\text{总量} = \text{人} \times \text{月}$ 。
- b. 盯紧问题最后的问句：
  - 看到「何人...」 → 立刻用  $\text{总量} \div \text{月份}$ 。
  - 看到「何ヶ月...」 → 立刻用  $\text{总量} \div \text{人数}$ 。
- c. 如果题目给的时间是“天数”（比如 95 日间），而人月算出来是“月”，别慌！考试通常会默认 1 个月 = 20 天或 25 天（题目会说明），或者像我们上一道题一样，总工数和总天数刚好能直接约分抵消。

45.